

WhatNext Vision Motors: Shaping the Future of Mobility with Innovation and Excellence

College Name:University College of Engineering Kanchipuram

Team No: 6

Team Lead Name: Prema U

Team Lead Mail id: premau2006@gmail.com

Team Member1

Name: Jeevitha R

Mail id: rjeevitha372@gmail.com

Team Member2

Name: Roopavathy G

Mail id: roopavathythilagavathi@gmail.com

Team Member3

Name: Ramya D

Mail id: ramyadhandapani9361@gmail.com

Team Member4

Name: Keerthika

Mail id: keerthikakk816@gmail.com

User Story:

WhatNext Vision Motors has implemented a Salesforce CRM solution to modernize its vehicle sales and customer management operations.

The system simplifies the vehicle ordering process and connects customers with suitable dealers based on their location.

It also ensures that customers cannot place orders for vehicles that are currently unavailable in stock.

Automated workflows help monitor order progress and update order statuses without requiring constant manual intervention.

Customers receive timely email reminders for their scheduled test drives.

Apex triggers are used to validate vehicle stock before processing customer orders.

Batch Apex jobs help maintain and update vehicle stock information efficiently.

Scheduled Apex automates routine order processing and other time-based activities.

Overall, the solution reduces manual work, improves order accuracy, enhances customer service, and increases operational efficiency.

Capability	Salesforce Feature Used
Nearest-dealer auto-assignment	Record-triggered Flow “Auto Assign Dealer” on Vehicle Order
Out-of-stock order prevention	Apex Trigger VehicleOrderTrigger + VehicleOrderTriggerHandler
Dynamic order status updates	Batch Apex VehicleOrderBatch (Pending → Confirmed)
Automated order processing	Scheduled Apex VehicleOrderBatchScheduler (daily)
Test drive email reminders	Record-triggered Flow “Test Drive Reminder” with a 1-day-before scheduled path
Operational visibility	Lightning App “WhatNext Vision Motors”, Reports and Dashboards

1. INTRODUCTION

1.1 Project Overview

WhatNext Vision Motors introduced a Salesforce CRM platform to enhance its vehicle sales and dealer management activities. The solution simplifies the purchasing process by automating order creation, checking vehicle inventory, directing customers’ orders to the closest suitable dealer, arranging test drive notifications, and maintaining real-time order status updates.

By bringing these activities together into one centralized system, the project eliminates manual handling and fragmented operations. The Salesforce implementation helps streamline business processes, reduce order-related issues, improve service coordination, and deliver a more convenient experience for customers.

1.2 Purpose

The main objective of this project is to streamline vehicle sales, inventory control, dealer allocation, and customer communication through Salesforce CRM. The system is designed to:

- Minimize human effort involved in vehicle orders and dealer-related activities
- Prevent booking issues resulting from incorrect or outdated vehicle stock details
- Enable live monitoring of vehicle inventory, order progress, and test drive schedules
- Improve operational performance while providing customers with a smooth and convenient experience

2. IDEATION PHASE

2.1 Problem Statement

The Customer Problem Statement Template below captures the two key stakeholders affected by the manual, disconnected vehicle ordering process.

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Operations Manager	manage vehicle orders	stock info is manual	systems are disconnected	stressed
PS-2	Customer	place vehicle orders	availability unclear	delayed updates	uncertain

Explanation: This table follows the Customer Problem Statement template. It records the two pain points that triggered the project: the Operations Manager (PS-1) cannot manage orders because stock information is manual and systems are disconnected, which causes stress; the Customer (PS-2) cannot tell whether a vehicle is available because updates are delayed, which causes uncertainty

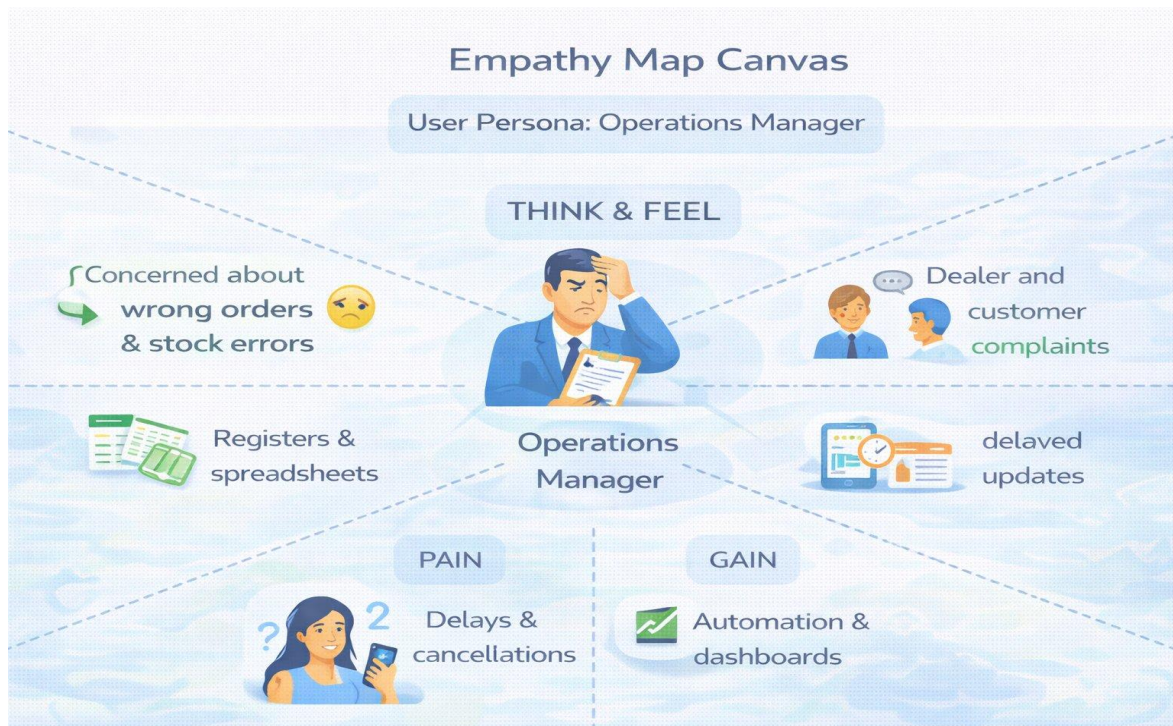


2.2 Empathy Map Canvas

User Persona: Operations Manager

Dimension	Insight
Think & Feel	Concerned about wrong orders & stock errors
Hear	Dealer and customer complaints
See	Registers & spreadsheets
Say & Do	Manual follow-ups
Pain	Delays & cancellations
Gain	Automation & dashboards

Explanation: The Empathy Map Canvas shows what the Operations Manager thinks, hears, sees, says and does. It highlights the pain (delays and cancellations) and the gain (automation and dashboards), which became the design goals for the CRM.



2.3 Brainstorming & Idea Prioritization

Ideas Considered

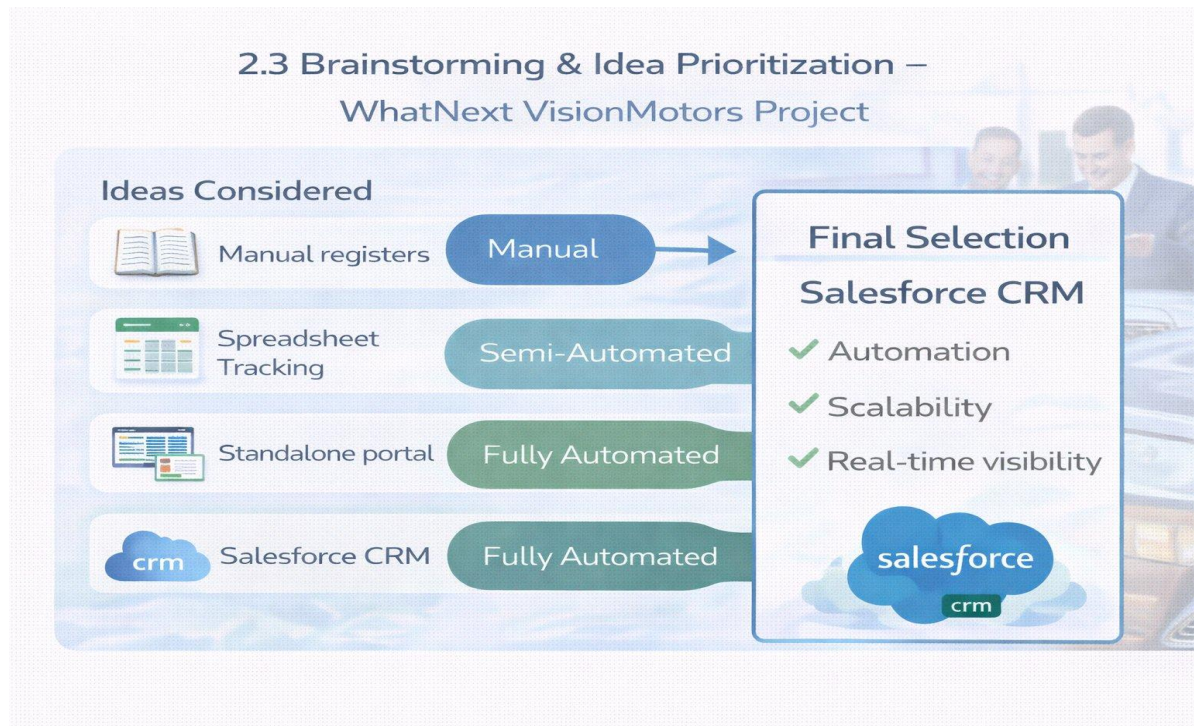
Idea	Automation Level	Outcome
Manual registers	Manual	Not selected
Spreadsheet tracking	Semi-Automated	Not selected
Standalone portal	Fully Automated	Not selected
Salesforce CRM	Fully Automated	Selected

Final Selection

Salesforce CRM was selected due to:

- Automation
- Scalability
- Real-time visibility

Explanation: Four approaches were compared: manual registers, spreadsheet tracking, a standalone portal and Salesforce CRM. Salesforce CRM was chosen because it offers automation, scalability and real-time visibility.



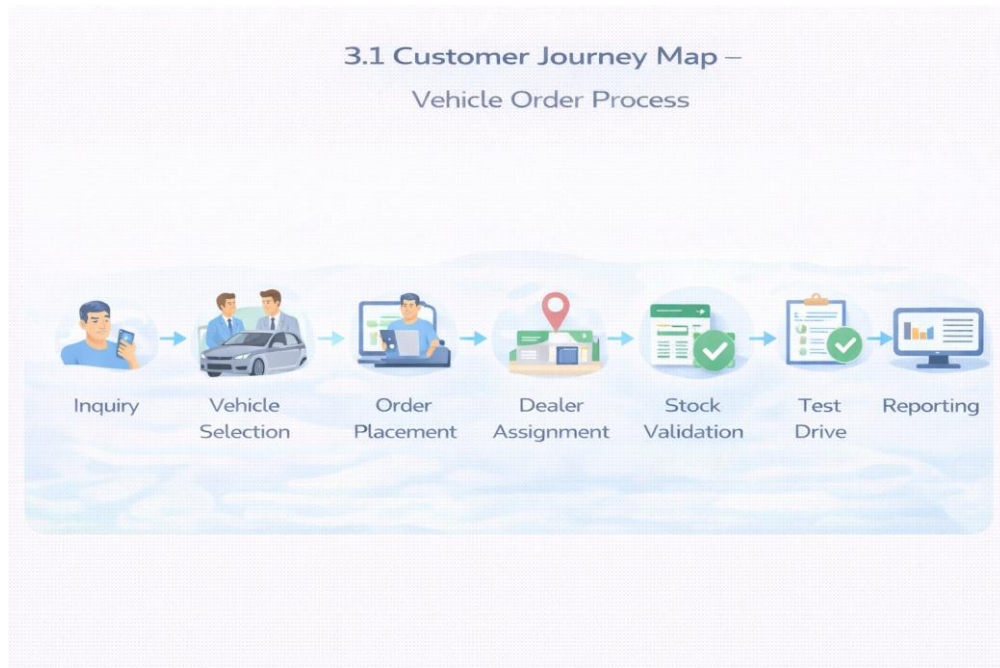
3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

The customer journey for the vehicle order process moves through the following stages:

Inquiry → Vehicle Selection → Order Placement → Dealer Assignment → Stock Validation → Test Drive → Confirmation → Reporting

Explanation: The Customer Journey Map shows the seven stages of a vehicle order, from Inquiry to Reporting. Each stage maps to a feature built later in the project, such as dealer assignment and stock validation.



3.2 Solution Requirements

Functional Requirements

FR No	Requirement	Realized Through
FR-1	Vehicle management	Vehicle__c object, tab and fields
FR-2	Dealer management	Vehicle_Dealer__c object, tab and fields
FR-3	Customer order creation	Vehicle_Customer__c and Vehicle_Order__c objects
FR-4	Stock validation	Apex trigger and handler class (VehicleOrderTrigger)
FR-5	Dealer auto-assignment	Record-triggered flow “Auto Assign Dealer”
FR-6	Test drive scheduling	Vehicle_Test_Drive__c object and “Test Drive Reminder” flow
FR-7	Reports & dashboards	Reports and Dashboards tabs in the Lightning App

Non-Functional Requirements

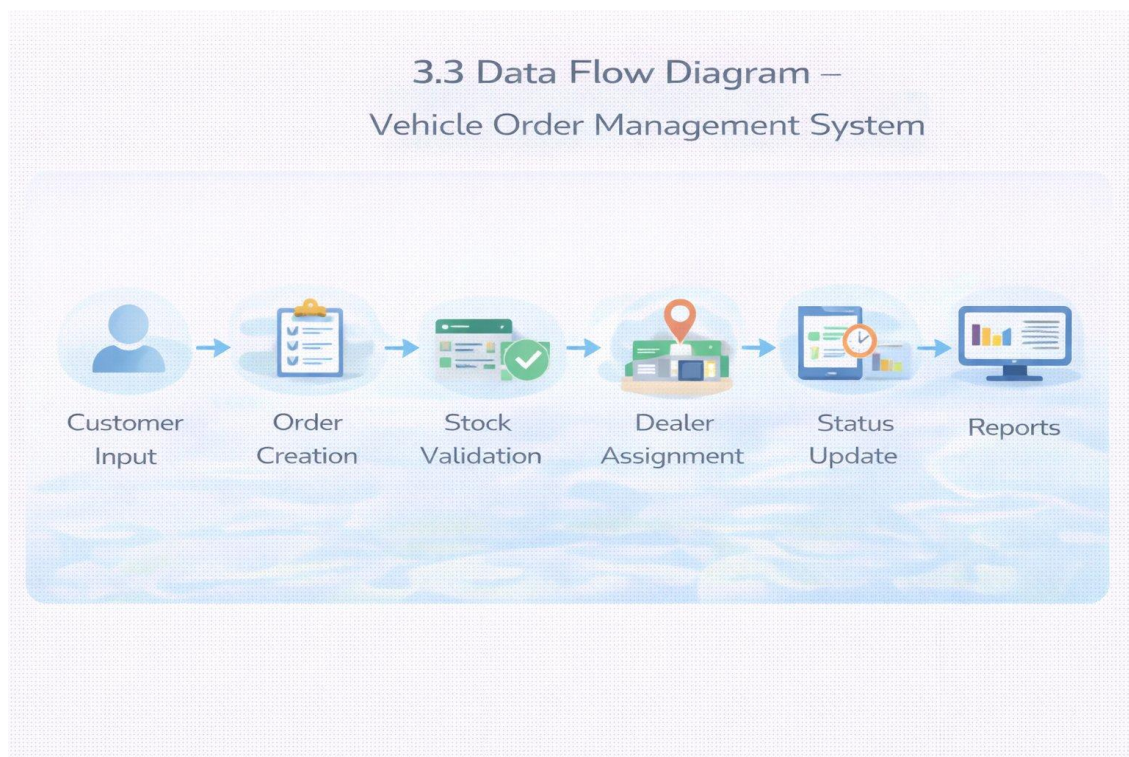
NFR	Description
Usability	Lightning UI for easy navigation
Security	Role-based access control
Reliability	Accurate automation
Performance	Fast order processing
Availability	24/7 cloud access
Scalability	Multi-dealer support

3.3 Data Flow Diagram

Data moves through the Vehicle Order Management System in the following sequence:

Customer Input → Order Creation → Stock Validation → Dealer Assignment → Status Update → Reports

Explanation: The Data Flow Diagram shows how data moves through the system: customer input creates an order, the order is checked against stock, a dealer is assigned, the status is updated and the results appear in reports.



3.4 Technology Stack

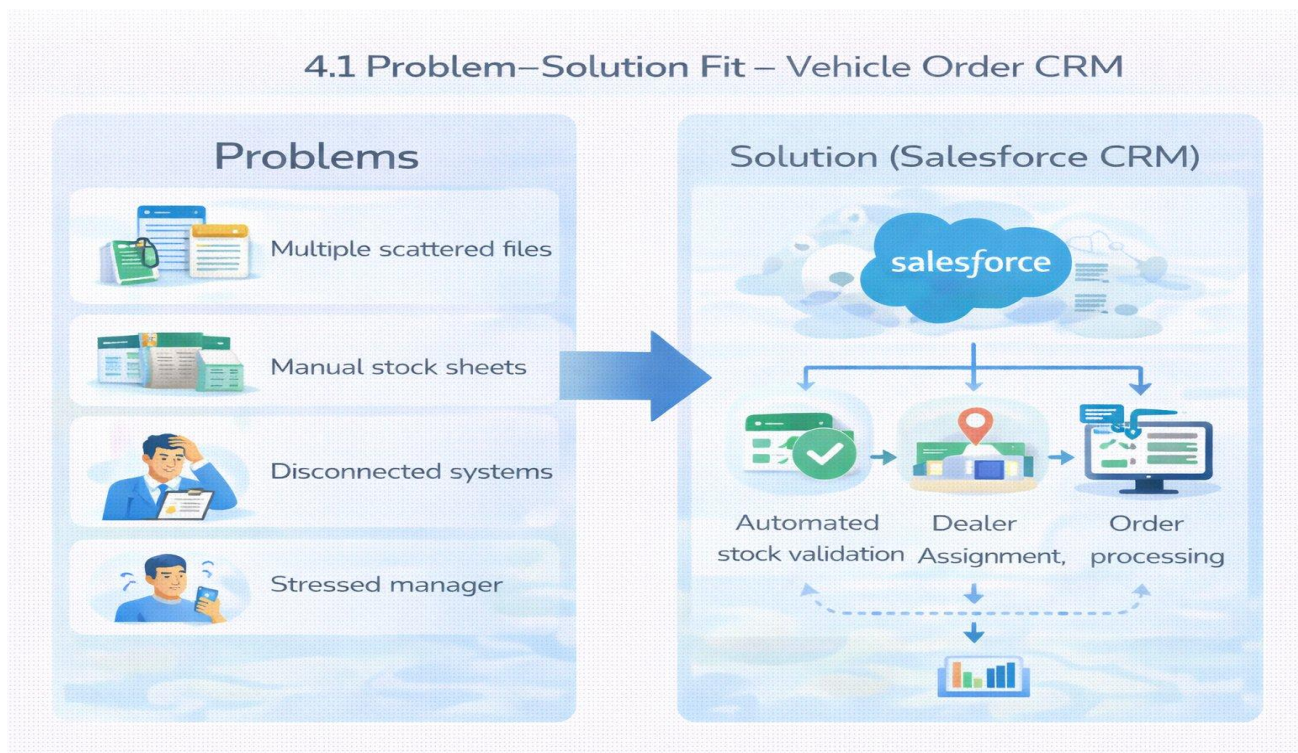
Layer	Technology
UI	Salesforce Lightning
Logic	Salesforce Flows
Database	Salesforce Objects
Automation	Validation Rules, Triggers
Processing	Batch Apex, Scheduled Apex
Reporting	Reports & Dashboards

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

The system eliminates fragmented operations by integrating automated stock validation, dealer assignment, and order processing within Salesforce.

Explanation: On the left are the problems: scattered files, manual stock sheets, disconnected systems and a stressed manager. On the right is the Salesforce solution: automated stock validation, dealer assignment and order processing, feeding reports.



4.2 Proposed Solution

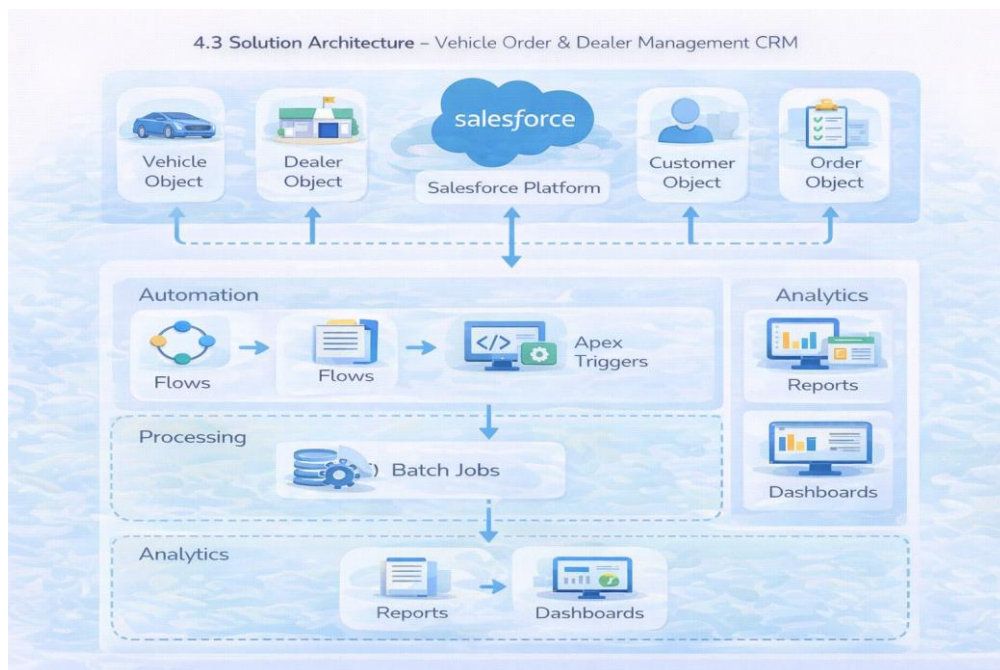
Parameter	Description
Problem	Manual vehicle ordering
Solution	Salesforce CRM
Innovation	Automated dealer assignment
Impact	Faster service
Business Model	Dealer network
Scalability	Multi-location support

4.3 Solution Architecture

The solution is composed of the following components:

Component	Role in the Solution
Vehicle Object	Stores vehicle details and stock quantity
Dealer Object	Stores authorized dealer information
Customer Object	Stores customer details
Order Object	Tracks vehicle purchases and order status
Test Drive Object	Tracks test drive bookings
Service Request Object	Tracks vehicle servicing requests
Flows	Dealer auto-assignment and test drive email reminders
Apex Triggers	Stock validation and stock update on order placement
Batch Jobs	Automatic order confirmation and stock updates
Reports	Order status, dealer performance and sales metrics
Dashboards	Visual monitoring of operations

Explanation: The solution architecture shows the data objects (Vehicle, Dealer, Customer, Order) on the Salesforce platform, with Flows and Apex Triggers for automation, Batch Jobs for processing, and Reports and Dashboards for analytics.



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

- Agile methodology
- Epics → Stories → Story Points
- Sprint-based execution
- Velocity calculation used for estimation

Product Backlog, Sprint Schedule, and Estimation

Sprint	Functional Requirement (Epic)	User Story No.	User Story / Task	Story Points	Priority	Team Member
Sprint-1	Developer Setup	USN-1	As a system administrator, I want to create and configure a Salesforce developer account so that I can build and deploy the vehicle order management system.	3	High	Member1

Sprint	Functional Requirement (Epic)	User Story No.	User Story / Task	Story Points	Priority	Team Member
Sprint-2	Data Modeling	USN-2	As an admin, I want to create Vehicle, Dealer, Customer, Order, and Service Request objects with proper relationships so that all business data is stored and managed systematically.	5	High	Member2
Sprint-2	Data Modeling	USN-3	As a staff member, I want dedicated tabs and a Lightning App for all modules so that I can easily navigate and manage vehicle orders and dealer operations.	5	High	Member3
Sprint-3	Automation	USN-4	As a system, I want to validate stock availability, mandatory fields, and order inputs so that invalid or incomplete vehicle orders are prevented.	3	High	Member4
Sprint-3	Automation	USN-5	As a sales coordinator, I want automated flows to assign the nearest dealer and update order status so that processing becomes faster and requires minimal manual effort.	3	High	Member5

Sprint	Functional Requirement (Epic)	User Story No.	User Story / Task	Story Points	Priority	Team Member
Sprint-4	Apex	USN-6	As a system, I want Apex triggers to automatically perform actions such as order confirmation and dealer assignment so that critical processes occur instantly and accurately.	5	High	Member1
Sprint-4	Apex	USN-7	As an inventory manager, I want batch jobs to update vehicle stock levels automatically so that inventory data remains accurate and up to date.	5	High	Member2
Sprint-5	Reports	USN-8	As a manager, I want reports showing order status, dealer performance, and sales metrics so that I can monitor operations and make informed decisions.	4	High	Member3
Sprint-6	Dashboards	USN-9	As a manager, I want dashboards showing order status, dealer performance, and sales metrics so that I can monitor operations and make informed decisions.	4	Medium	Member4

Project Tracker, Velocity & Burndown

Milestone	Main Deliverable
Milestone 1	Salesforce Developer Account
Milestone 2	Objects & Relationships
Milestone 3	Tabs
Milestone 4	Lightning App
Milestone 5	Fields & Relationships
Milestone 6	Flows
Milestone 7	Apex, Trigger and Batch Jobs

6. PROJECT DEVELOPMENT PHASE

Phase 1: Requirement Analysis & Planning

Objective

The project objective is to create a Salesforce-based automotive ordering solution that improves dealer assignment, validates vehicle stock, automates test-drive reminders and supports scheduled order processing.

Key Requirements

- Manage vehicle and dealer information.
- Manage customer, order, test-drive and service-request records.
- Assign a dealer based on customer address.
- Prevent orders when vehicle stock is unavailable.
- Send reminders for scheduled test drives.
- Process pending orders when stock becomes available.

Defining Project Scope & Objectives

- Centralize automotive records in Salesforce.
- Support the vehicle ordering lifecycle.
- Automate dealer assignment and test-drive reminders.
- Use Apex for stock validation and stock updates.
- Use Batch Apex and Scheduled Apex for periodic processing.

Gathering & Analysing User Needs

User	Key Need
Customer	Order a vehicle, receive suitable dealer assignment and manage test drives.
Sales/Operations Staff	Process orders and maintain accurate stock information.
Administrator	Configure Salesforce objects, fields, automation and access.
Dealer/Operations	Work with order information associated with vehicle availability.

Tools Used

- Salesforce Developer Org
- Object Manager
- Lightning App Builder
- Flow Builder
- Developer Console
- Apex
- Batch Apex
- Scheduled Apex

Phase 2: Salesforce Development – Backend & Configuration

Milestone 1: Salesforce Account

Activity 1: Creating Developer Account

Create a developer org in Salesforce:

1. Go to <https://developer.salesforce.com/signup>
2. On the sign-up form, enter the following details: First name & Last name; Email; Role: **Developer**; Company: **College Name**; Country: **India**; Postal Code: **pin code**; Username: a combination of your name and company, in the format `username@organization.com` (it need not be an actual email id).
3. Click **Sign me up** after filling in these details.

Build for the Agentic Enterprise. Sign up for your Developer Edition.

The entire Salesforce platform - exposed as APIs, MCP tools, and CLI commands. Build apps, ship agents, and let coding agents do the heavy lifting with Salesforce Headless 360.

- Build AI agents with Agentforce, grounded in your data with Data 360
- Connect Claude Code, Cursor, or Codex to your org with hosted MCP
- Build in the browser with the Agentforce Vibes IDE - no setup required
- Drag-and-drop when you want it, Apex when you need it
- 60+ MCP tools and 30+ coding skills, ready out of the box

Sign up for your Developer Edition

A free Salesforce Platform environment with Agentforce and Data 360

First name

Prema Jeevitha ✓

Last name

Roopavathy Ramya Keerthi... ✓

Job title

Developer ✓

Company

University college Of Engin... ✓

Country/region

India ✓

Work email

premau2006@gmail.com ✓

☒ I agree to the [Main Services Agreement – Developer Services](#), [Salesforce Program Agreement](#), and [API Terms of Service](#)

Your free trial may be provisioned on or migrated to Hyperforce, Salesforce's public cloud infrastructure.

I acknowledge, as described in the Developer Documentation: (1) the Developer Edition includes autonomous and other generative AI features; and (2) Salesforce may limit use of those features and the org, and may terminate any org that has been inactive for 45 days.

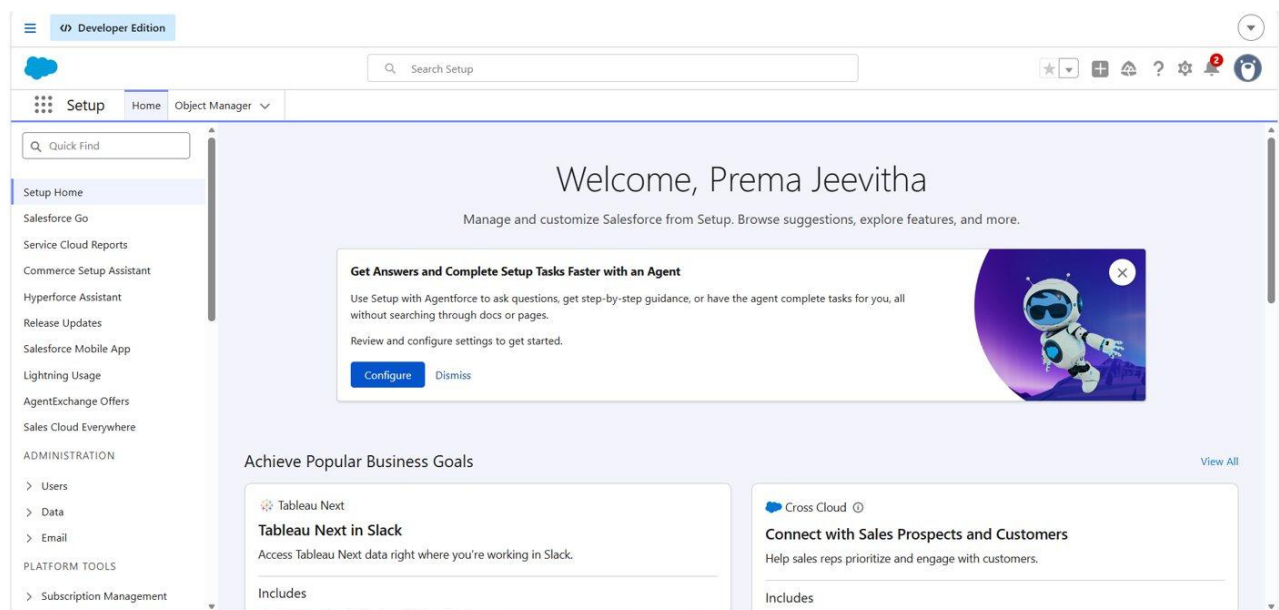
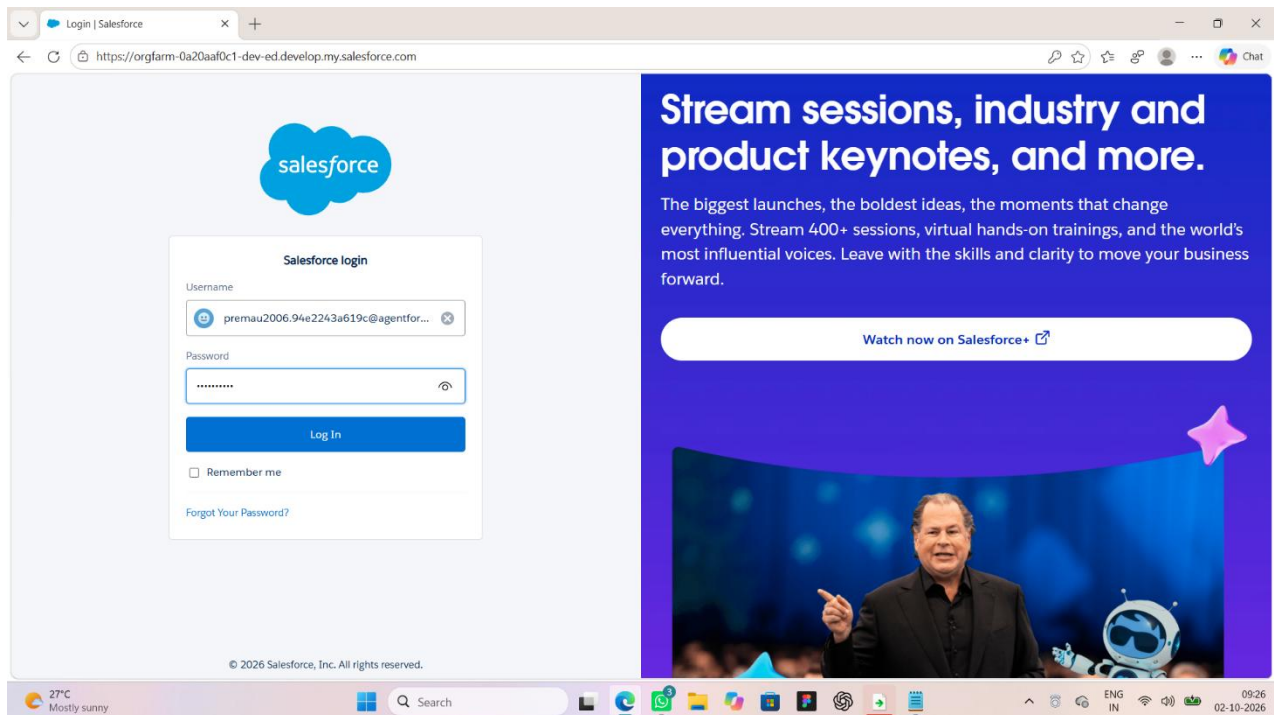
We value your privacy. To learn more, visit our [Privacy Statement](#).

Sign me up

Activity 2: Account Activation

1. Go to the inbox of the email used while signing up and click **Verify Account** to activate the account. The email may take 5–10 minutes.
2. Provide a password, answer a security question and click **Change Password**.
3. You are then redirected to your Salesforce Setup page.

Explanation: This is the Setup Home page of the Salesforce Developer Edition org after the account was created and verified. It confirms the org is ready to build the vehicle order management system.



Milestone 2: Data Management – Objects

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle

To Navigate to the Setup Page

Click on the gear icon → click **Setup**.

Activity 1: Create Vehicle Object

The purpose of creating a Vehicle custom object is to have a clear picture of the vehicle details. To create an object:

1. From the Setup page → click **Object Manager** → click **Create** → click **Custom Object**.
2. Enter the label name → **Vehicle**; Plural label name → **Vehicles**.
3. Enter the Record Name Label and Format: Record Name → **Vehicle Name**; Data Type → **Text**.
4. Click **Allow Reports**.
5. Select **Allow Search** → **Save**.

Activity 2: Create Dealer Object

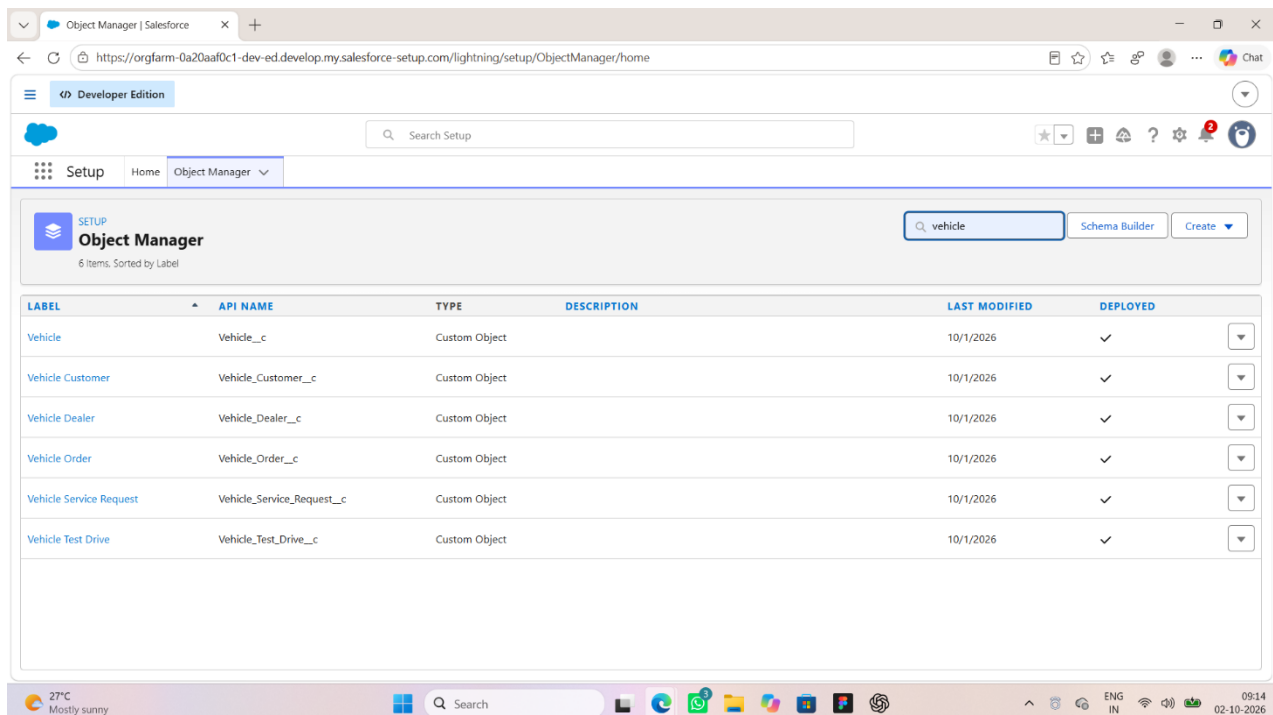
The purpose of creating a Dealer object is to have detailed information about the dealer. To create an object:

1. From the Setup page → click **Object Manager** → click **Create** → click **Custom Object**.
2. Enter the label name → **Vehicle Dealer**; Plural label name → **Vehicle Dealers**.
3. Enter the Record Name Label and Format: Record Name → **Dealer Name**; Data Type → **Text**.
4. Click **Allow Reports**, then **Allow Search** → **Save**.

Activity 3: Creating Remaining Objects

Create the remaining objects mentioned in the Data Management – Objects table above: Vehicle Customer, Vehicle Order, Vehicle Test Drive and Vehicle Service Request.

Explanation: The Object Manager is filtered by “vehicle” and lists the six custom objects created for this project: Vehicle, Vehicle Customer, Vehicle Dealer, Vehicle Order, Vehicle Service Request and Vehicle Test Drive. All are custom objects and are deployed.



The screenshot shows the Salesforce Object Manager interface. The search bar at the top right contains the text "vehicle". Below the search bar, there is a table with 6 items, sorted by Label. The table has columns for LABEL, API NAME, TYPE, DESCRIPTION, LAST MODIFIED, and DEPLOYED. The objects listed are Vehicle, Vehicle Customer, Vehicle Dealer, Vehicle Order, Vehicle Service Request, and Vehicle Test Drive. All are Custom Objects, created on 10/1/2026, and are deployed.

LABEL	API NAME	TYPE	DESCRIPTION	LAST MODIFIED	DEPLOYED
Vehicle	Vehicle__c	Custom Object		10/1/2026	✓
Vehicle Customer	Vehicle_Customer__c	Custom Object		10/1/2026	✓
Vehicle Dealer	Vehicle_Dealer__c	Custom Object		10/1/2026	✓
Vehicle Order	Vehicle_Order__c	Custom Object		10/1/2026	✓
Vehicle Service Request	Vehicle_Service_Request__c	Custom Object		10/1/2026	✓
Vehicle Test Drive	Vehicle_Test_Drive__c	Custom Object		10/1/2026	✓

Milestone 3: Data Management – Tabs

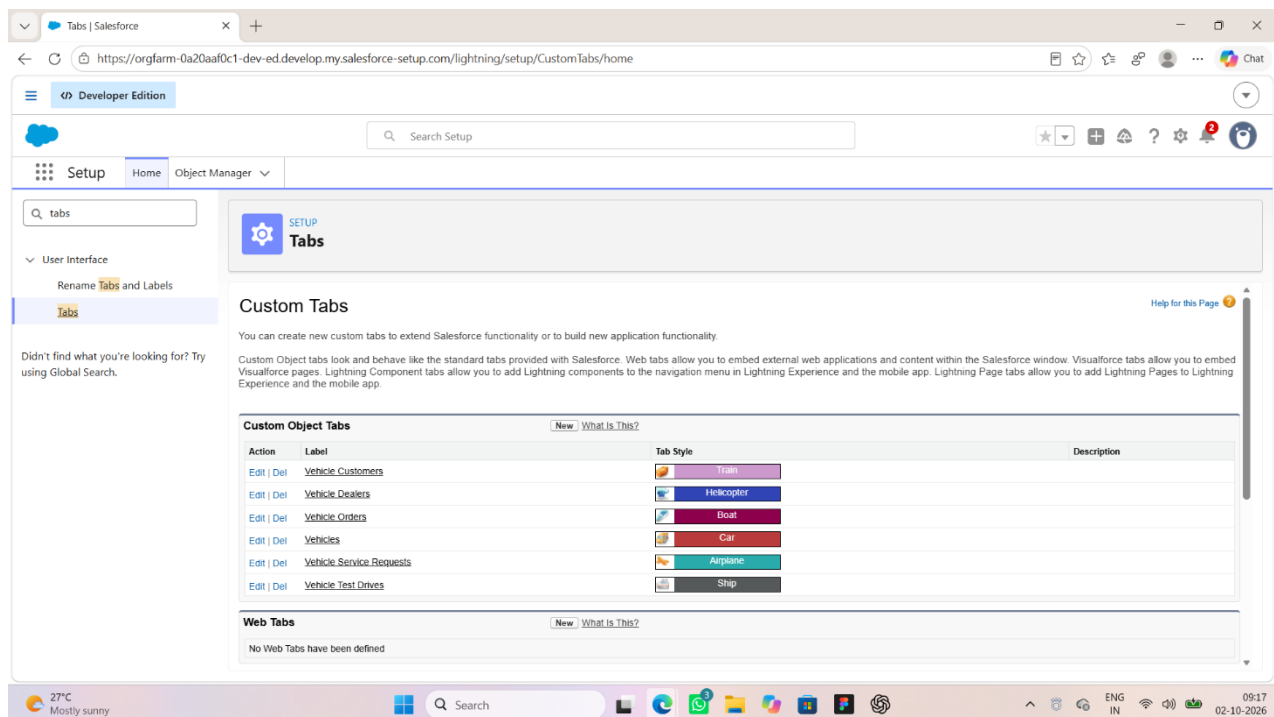
Activity 1: Creating a Custom Tab (Vehicle)

1. Go to the Setup page → type **Tabs** in the Quick Find bar → click **Tabs** → **New** (under Custom Object Tabs).
2. Select Object (**Vehicle**) → select any tab style → **Next** (Add to profiles page) keep it as default → **Next** (Add to Custom App) keep it as default → **Save**.

Activity 2: Tabs for the Other Objects

Create tabs for the other objects as well, using the same procedure.

Explanation: The Custom Object Tabs section lists a tab for every object (Vehicles, Vehicle Customers, Vehicle Dealers, Vehicle Orders, Vehicle Service Requests and Vehicle Test Drives), so users can open each module directly.

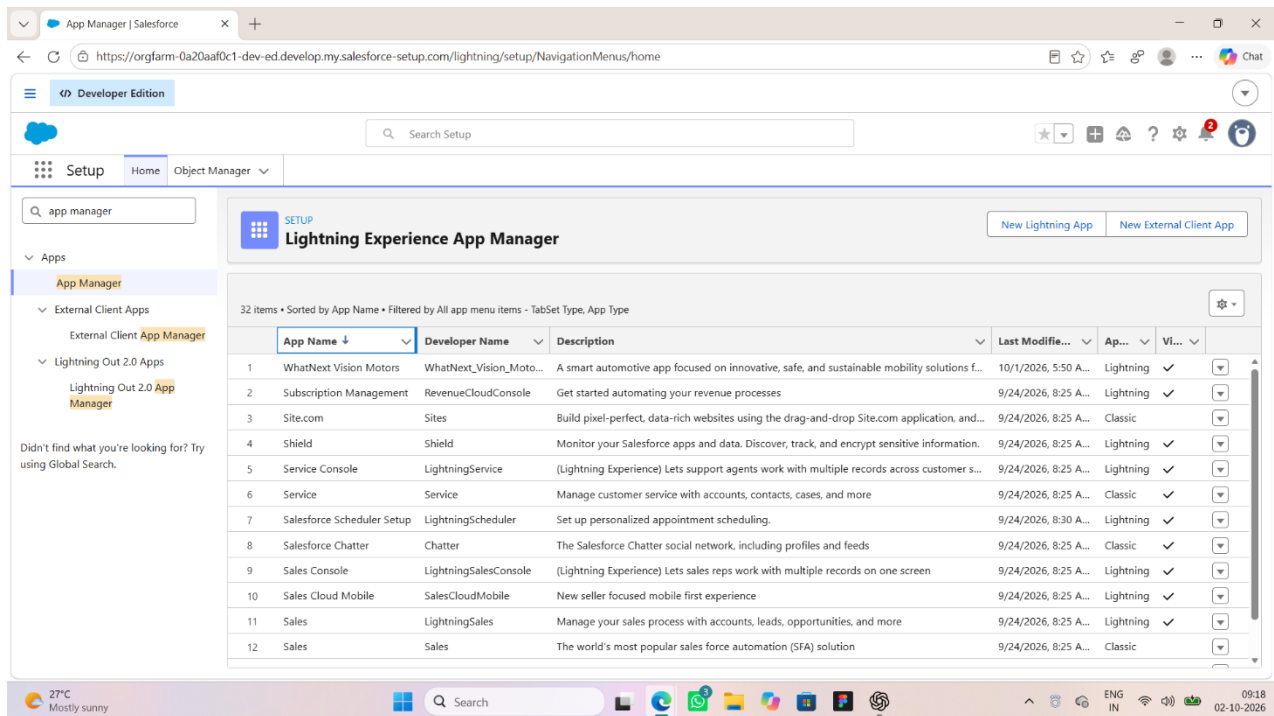


Milestone 4: Data Management – App Manager

Activity 1: Create a Lightning App

1. Go to the Setup page → search “app manager” in Quick Find → select **App Manager** → click **New Lightning App**.
2. Fill in the app details and branding: App Name: **WhatNext Vision Motors**; Developer Name: auto-populated; Description: a meaningful description; Image: optional; Primary color hex value: keep the default.
3. Click **Next** → (App Options page) keep the default → **Next** → (Utility Items) keep the default → **Next**.
4. Add Navigation Items: search for Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports and Dashboard in the search bar and move them using the arrow button → **Next**. Select the custom objects created in the previous milestone.
5. Add User Profiles: search for **System Administrator**, click the arrow button → **Save & Finish**.

Explanation: The Lightning Experience App Manager lists the new Lightning app “WhatNext Vision Motors”, with its developer name and description, confirming the app was created.



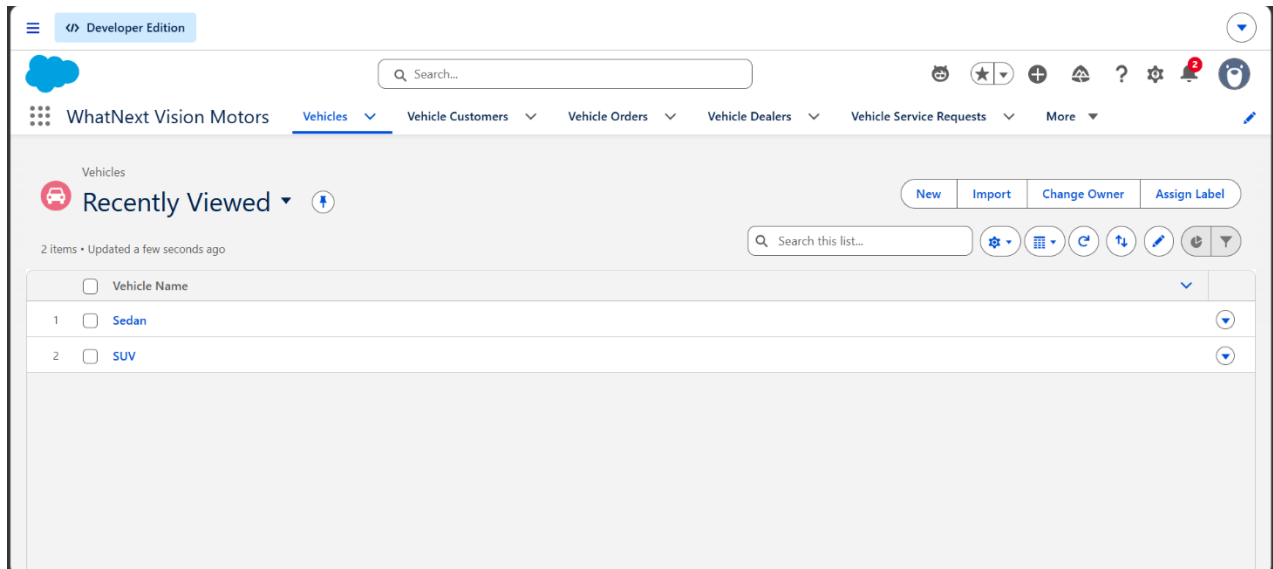
The screenshot shows the Salesforce Lightning Experience App Manager interface. The left sidebar contains a navigation menu with 'App Manager' selected. The main content area displays a table of 32 items, sorted by App Name. The table includes columns for App Name, Developer Name, Description, Last Modified, App Type, and Visibility. The first item in the list is 'WhatNext Vision Motors' by 'WhatNext_Vision_Moto...'. Below the table, there is a search bar and a 'New Lightning App' button.

	App Name	Developer Name	Description	Last Modified	App Type	Visibility
1	WhatNext Vision Motors	WhatNext_Vision_Moto...	A smart automotive app focused on innovative, safe, and sustainable mobility solutions f...	10/1/2026, 5:50 A...	Lightning	✓
2	Subscription Management	RevenueCloudConsole	Get started automating your revenue processes	9/24/2026, 8:25 A...	Lightning	✓
3	Site.com	Sites	Build pixel-perfect, data-rich websites using the drag-and-drop Site.com application, and...	9/24/2026, 8:25 A...	Classic	
4	Shield	Shield	Monitor your Salesforce apps and data. Discover, track, and encrypt sensitive information.	9/24/2026, 8:25 A...	Lightning	✓
5	Service Console	LightningService	(Lightning Experience) Lets support agents work with multiple records across customer s...	9/24/2026, 8:25 A...	Lightning	✓
6	Service	Service	Manage customer service with accounts, contacts, cases, and more	9/24/2026, 8:25 A...	Classic	✓
7	Salesforce Scheduler Setup	LightningScheduler	Set up personalized appointment scheduling.	9/24/2026, 8:30 A...	Lightning	✓
8	Salesforce Chatter	Chatter	The Salesforce Chatter social network, including profiles and feeds	9/24/2026, 8:25 A...	Classic	✓
9	Sales Console	LightningSalesConsole	(Lightning Experience) Lets sales reps work with multiple records on one screen	9/24/2026, 8:25 A...	Lightning	✓
10	Sales Cloud Mobile	SalesCloudMobile	New seller focused mobile first experience	9/24/2026, 8:25 A...	Lightning	✓
11	Sales	LightningSales	Manage your sales process with accounts, leads, opportunities, and more	9/24/2026, 8:25 A...	Lightning	✓
12	Sales	Sales	The world's most popular sales force automation (SFA) solution	9/24/2026, 8:25 A...	Classic	

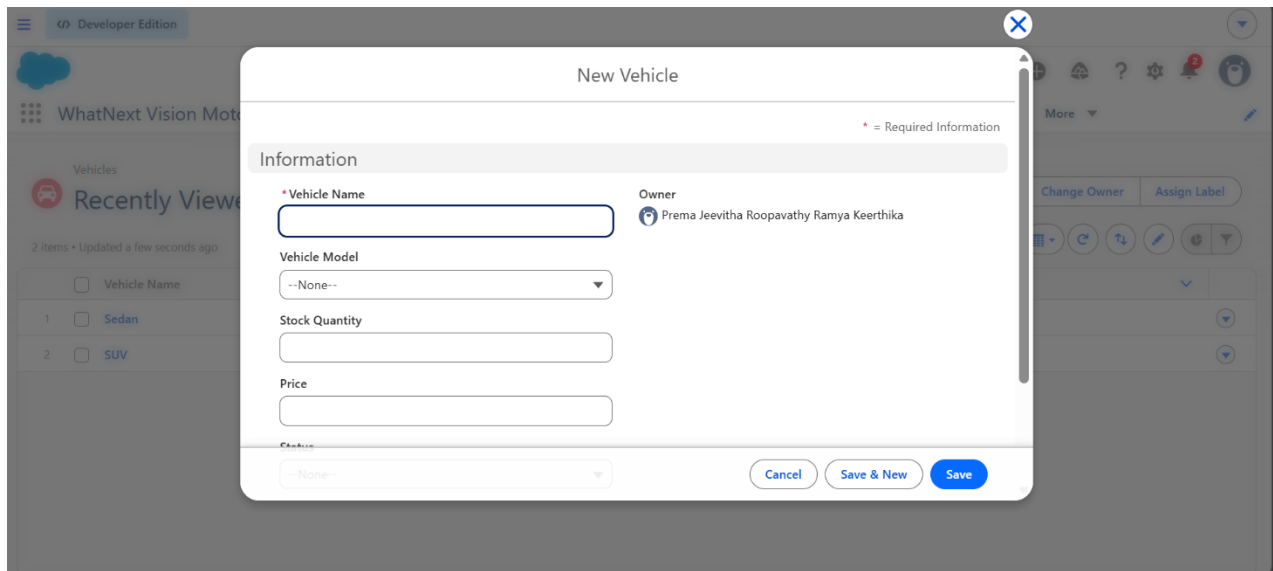


Logo: WhatNext Vision Motors

Explanation: This is the WhatNext Vision Motors Lightning app in use. The navigation bar shows the tabs for Vehicles, Vehicle Customers, Vehicle Orders, Vehicle Dealers, Vehicle Service Requests, Vehicle Test Drives, Reports and Dashboards. The Vehicle Customers list view shows two saved customer records.



ScreenShot:WhatNext Vision Motors



ScreenShot:WhatNext Vision Motors

The screenshot displays the 'Vehicle' details page for an SUV. The interface includes a top navigation bar with a search bar and various icons. The main content area shows the vehicle's details under the 'Details' tab. The details are organized into two columns: 'Vehicle Name' and 'Owner'.

Vehicle Name	Owner
SUV	Prema Jeevitha Roopavathy Ramya Keerthika
Vehicle Model	
SUV	
Stock Quantity	
4	
Price	
\$500,000	
Status	
Available	
Dealer	
Suriya	

ScreenShot:WhatNext Vision Motors

The screenshot displays the 'Vehicle Customers' page, specifically the 'Recently Viewed' section. The interface includes a top navigation bar with a search bar and various icons. The main content area shows a list of customers under the 'Recently Viewed' tab. The list is organized into two columns: 'Customer Name' and 'Owner'.

Customer Name	Owner
1 Divakar	
2 Dinesh	

ScreenShot:WhatNext Vision Motors

The screenshot displays the 'Vehicle Customer' profile for 'Dinesh'. The interface includes a top navigation bar with a search bar and various icons. The main menu shows 'Vehicle Customers' as the active section. The profile details are as follows:

Related	Details
Customer Name	Dinesh
Email	dinesh12345@gmail.com
Phone	(912) 345-6780
Address	Bangalore, Karnataka, India
Preferred Vehicle Type	Sedan
Created By	Prema Jeevitha Roopavathy Ramya Keerthika, 10/1/2026, 8:10 PM
Owner	Prema Jeevitha Roopavathy Ramya Keerthika
Last Modified By	Prema Jeevitha Roopavathy Ramya Keerthika, 10/1/2026, 8:10 PM

Buttons for 'New Contact', 'Edit', and 'New Opportunity' are visible in the top right corner of the profile view.

ScreenShot:WhatNext Vision Motors

The screenshot displays the 'Vehicle Orders' section. The interface includes a top navigation bar with a search bar and various icons. The main menu shows 'Vehicle Orders' as the active section. The 'Recently Viewed' list contains one item:

Order Name
1 SUV Order

Buttons for 'New', 'Import', 'Change Owner', and 'Assign Label' are visible in the top right corner of the list view. A search bar for the list is also present.

ScreenShot:WhatNext Vision Motors

This screenshot shows the 'SUV Order' details page in the WhatNext Vision Motors application. The page is divided into a left sidebar with navigation links (Vehicles, Vehicle Customers, Vehicle Orders, Vehicle Dealers, Vehicle Service Requests, More) and a main content area. The 'Vehicle Orders' section is active, displaying the 'SUV Order' details. The details include: Order Name (SUV Order), Vehicle Customer (Divakar), Vehicle (SUV), Order Date (10/20/2026), Status (Confirmed), Created By (Prema Jeevitha Roopavathy Ramya Keerthika), and Last Modified By (Prema Jeevitha Roopavathy Ramya Keerthika). The 'Details' tab is selected, and the 'Related' tab is also visible. The top navigation bar includes a search bar and various utility icons.

Field	Value
Order Name	SUV Order
Vehicle Customer	Divakar
Vehicle	SUV
Order Date	10/20/2026
Status	Confirmed
Created By	Prema Jeevitha Roopavathy Ramya Keerthika, 10/1/2026, 8:18 PM
Last Modified By	Prema Jeevitha Roopavathy Ramya Keerthika, 10/1/2026, 8:18 PM

ScreenShot:WhatNext Vision Motors

This screenshot shows the 'Recently Viewed' page for vehicle dealers in the WhatNext Vision Motors application. The page is divided into a left sidebar with navigation links (Vehicles, Vehicle Customers, Vehicle Orders, Vehicle Dealers, Vehicle Service Requests, More) and a main content area. The 'Vehicle Dealers' section is active, displaying a list of recently viewed dealers. The list includes two items: 'Suriya' and 'Sowmiya'. The 'Recently Viewed' tab is selected, and the 'New' tab is also visible. The top navigation bar includes a search bar and various utility icons.

Dealer Name
1 Suriya
2 Sowmiya

ScreenShot:WhatNext Vision Motors

The screenshot displays the 'Vehicle Dealers' page for a dealer named 'Suriya'. The interface includes a top navigation bar with a search bar and various icons. The main content area shows the dealer's details under the 'Details' tab. The details are organized into two columns: the left column contains fields for Dealer Name, Dealer Location, Dealer Code, Phone, Email, and Created By; the right column contains fields for Owner and Last Modified By. The dealer's information is as follows:

Field	Value
Dealer Name	Suriya
Dealer Location	Chennai, Tamil Nadu
Dealer Code	1
Phone	(987) 654-3210
Email	suriyadealer@gmail.com
Created By	Prema Jeevitha Roopavathy Ramya Keerthika, 10/1/2026, 8:16 PM
Owner	Prema Jeevitha Roopavathy Ramya Keerthika
Last Modified By	Prema Jeevitha Roopavathy Ramya Keerthika, 10/1/2026, 8:16 PM

The bottom of the screen shows a Windows taskbar with the date and time as 9:19 PM on 10/2/2026.

ScreenShot:WhatNext Vision Motors

The screenshot displays the 'Vehicle Service Requests' page. The interface includes a top navigation bar with a search bar and various icons. The main content area shows a list of service requests under the 'Recently Viewed' tab. The list contains one item, 'SUV Service Request', which is marked as 'Updated a few seconds ago'. The list is displayed in a table format with a search bar and various action buttons (New, Import, Change Owner, Assign Label) at the top. The table has a header row with a checkbox and a column for 'Service Request Name'. The data row shows a single entry with a checkbox and the text 'SUV Service Request'.

Service Request Name
SUV Service Request

ScreenShot:WhatNext Vision Motors

The screenshot displays the Salesforce Lightning interface for 'WhatNext Vision Motors'. The top navigation bar includes a search bar and a menu with options like 'Vehicles', 'Vehicle Customers', 'Vehicle Orders', 'Vehicle Dealers', 'Vehicle Service Requests', 'Reports', and 'More'. The 'Vehicle Service Requests' section is active, showing a record for 'SUV Service Request'. The record details include:

- Service Request Name:** SUV Service Request
- Vehicle Customer:** Divakar
- Vehicle:** SUV
- Service Date:** 10/15/2026
- Issue Description:** Engine oil change and general service required
- Status:** In Progress
- Created By:** (User icon)
- Owner:** Prema Jeevitha Roopavathy Ramya Keerthika
- Last Modified By:** (User icon)

Buttons for 'New Contact', 'Edit', and 'New Opportunity' are visible in the top right corner of the record view.

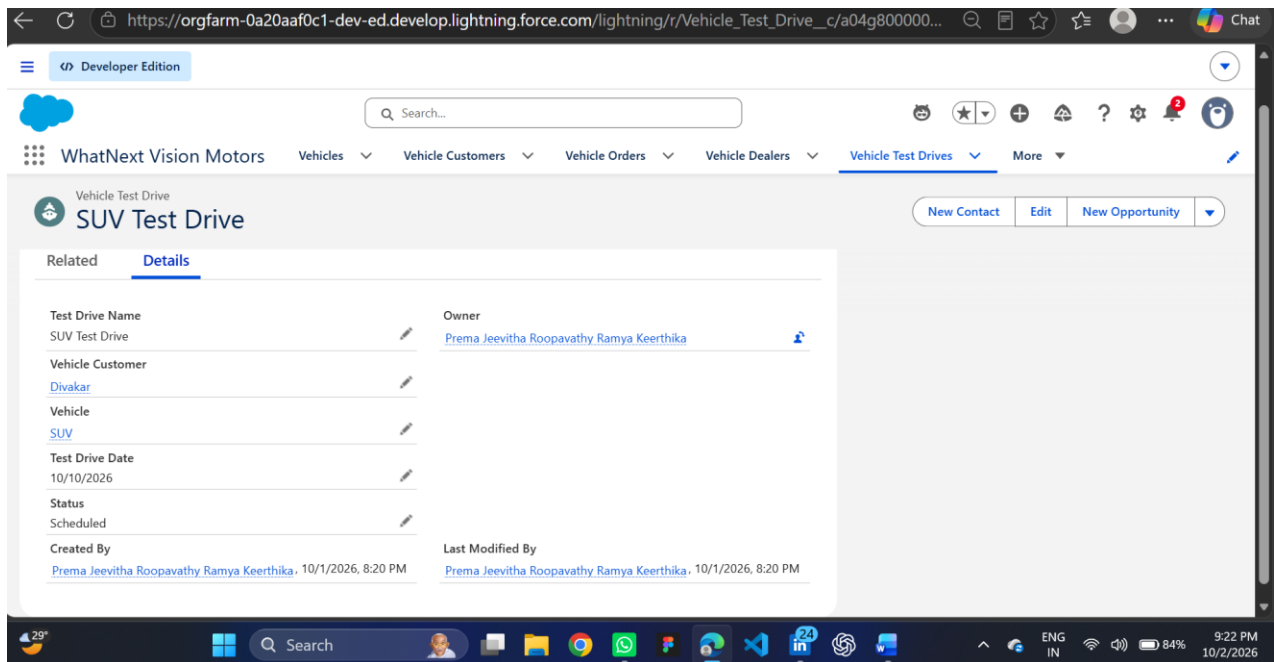
ScreenShot:WhatNext Vision Motors

The screenshot displays the Salesforce Lightning interface for 'WhatNext Vision Motors', showing a list of 'Vehicle Test Drives'. The top navigation bar is similar to the previous screenshot, with the 'Vehicle Test Drives' section active. The list view shows:

- Vehicle Test Drives** (Section Header)
- Recently Viewed** (Filter)
- 1 item • Updated a few seconds ago** (Status)
- Search this list...** (Search bar)
- Test Drive Name** (Column Header)
- 1 SUV Test Drive** (List Item)

Buttons for 'New', 'Import', 'Change Owner', and 'Assign Label' are visible in the top right corner of the list view.

ScreenShot:WhatNext Vision Motors



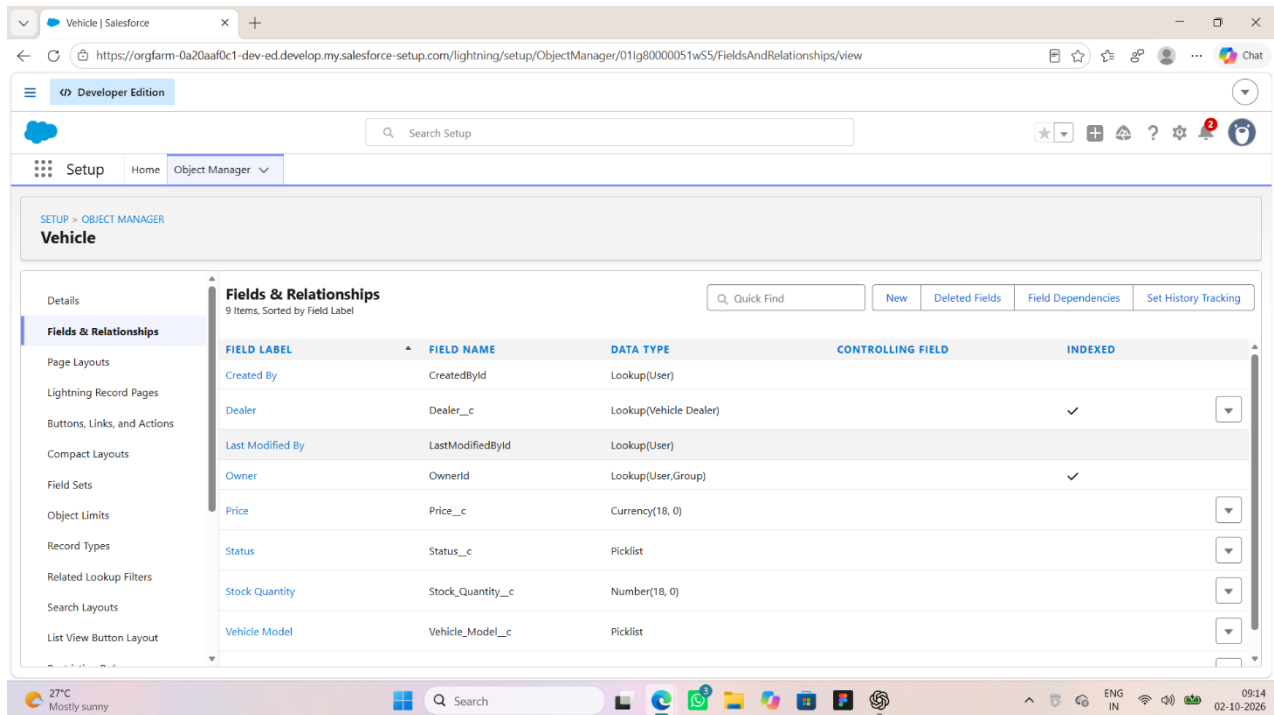
Milestone 5: Data Management – Fields

Key Fields for Each Object

1. Vehicle__c (Custom Object)

Field	Data Type	Details
Vehicle_Name__c	Text	Record name
Vehicle_Model__c	Picklist	Sedan, SUV, EV, etc.
Stock_Quantity__c	Number	Available stock count
Price__c	Currency	Vehicle price
Dealer__c	Lookup	Lookup to Vehicle_Dealer__c
Status__c	Picklist	Available, Out of Stock, Discontinued

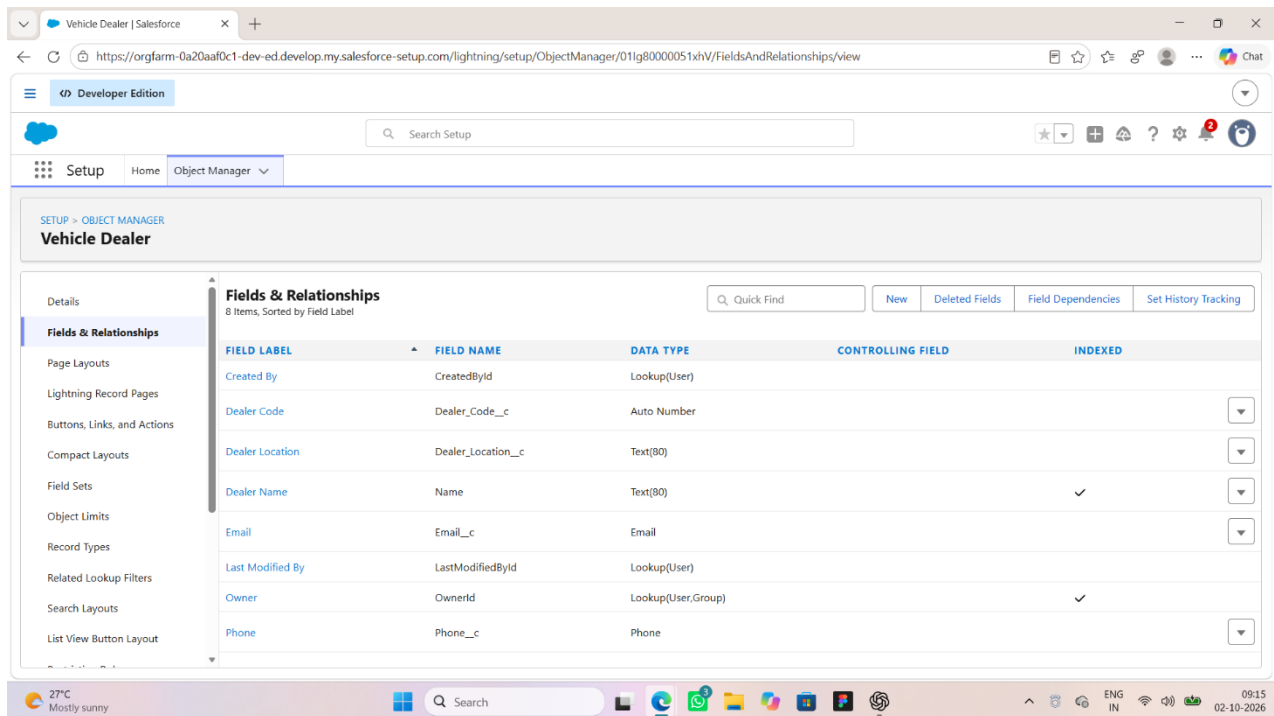
Explanation: The Vehicle object's fields: Dealer (lookup to Vehicle Dealer), Price (currency), Status (picklist), Stock Quantity (number) and Vehicle Model (picklist). Stock Quantity is the field the Apex trigger checks.



2. Vehicle_Dealer__c (Custom Object)

Field	Data Type	Details
Dealer_Name__c	Text	Record name
Dealer_Location__c	Text	Dealer location (used for nearest-dealer matching)
Dealer_Code__c	Auto Number	System-generated dealer code
Phone__c	Phone	Dealer contact number
Email__c	Email	Dealer email address

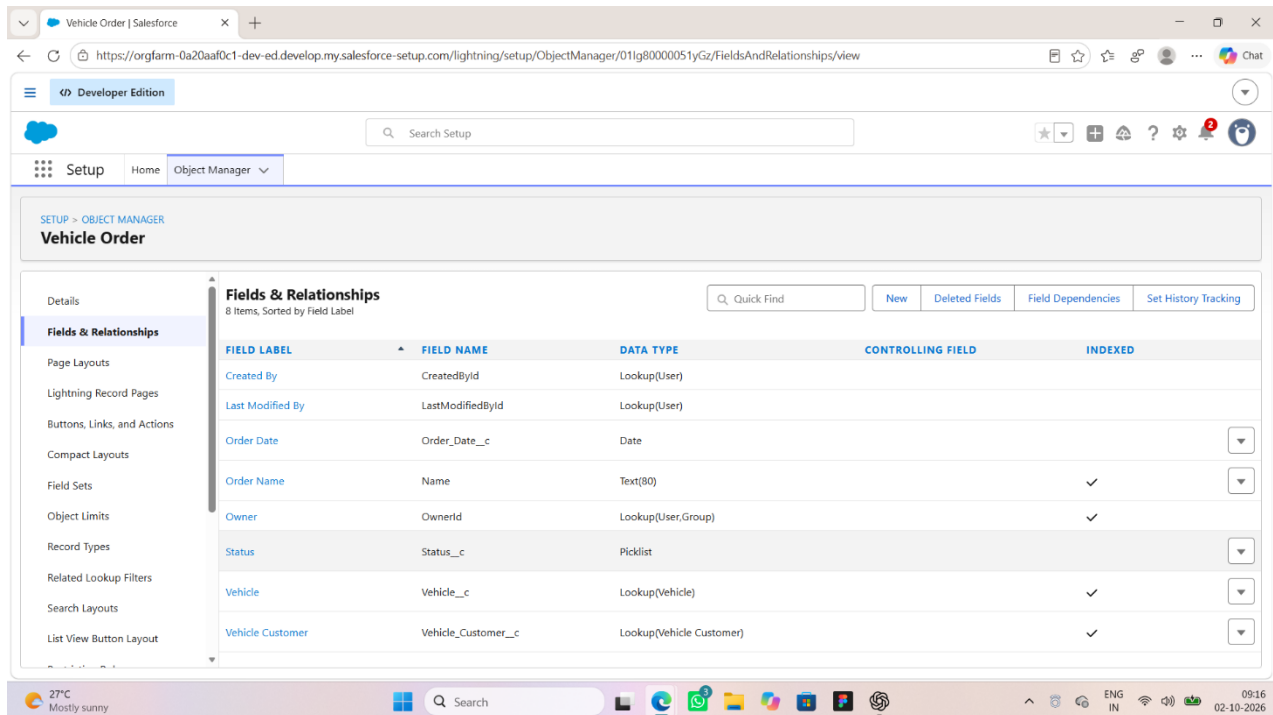
Explanation: The Vehicle Dealer object's fields: Dealer Code (auto number), Dealer Location (text), Dealer Name, Email and Phone. Dealer Location is the field the Auto Assign Dealer flow matches against the customer's address.



3. Vehicle_Order__c (Custom Object)

Field	Data Type	Details
Vehicle_Customer__c	Lookup	Lookup to Vehicle_Customer__c
Vehicle__c	Lookup	Lookup to Vehicle__c
Order_Date__c	Date	Date of the order
Status__c	Picklist	Pending, Confirmed, Delivered, Canceled

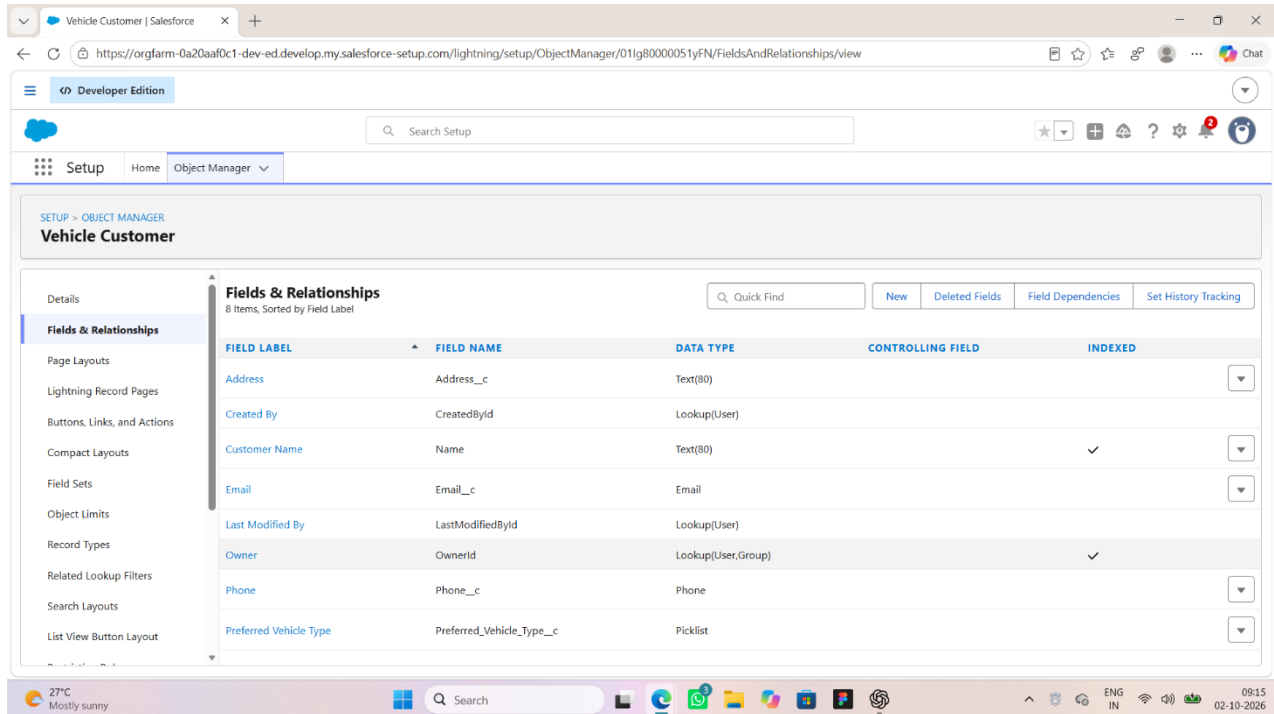
Explanation: The Vehicle Order object's fields: Order Date, Status (picklist), and lookups to Vehicle and Vehicle Customer. Status drives the flow, the trigger and the batch job.



4. Vehicle_Customer__c (Custom Object)

Field	Data Type	Details
Customer_Name__c	Text	Record name
Email__c	Email	Customer email address (used for test drive reminders)
Phone__c	Phone	Customer contact number
Address__c	Text	Customer address (used for dealer assignment)
Preferred_Vehicle_Type__c	Picklist	Sedan, SUV, EV, etc.

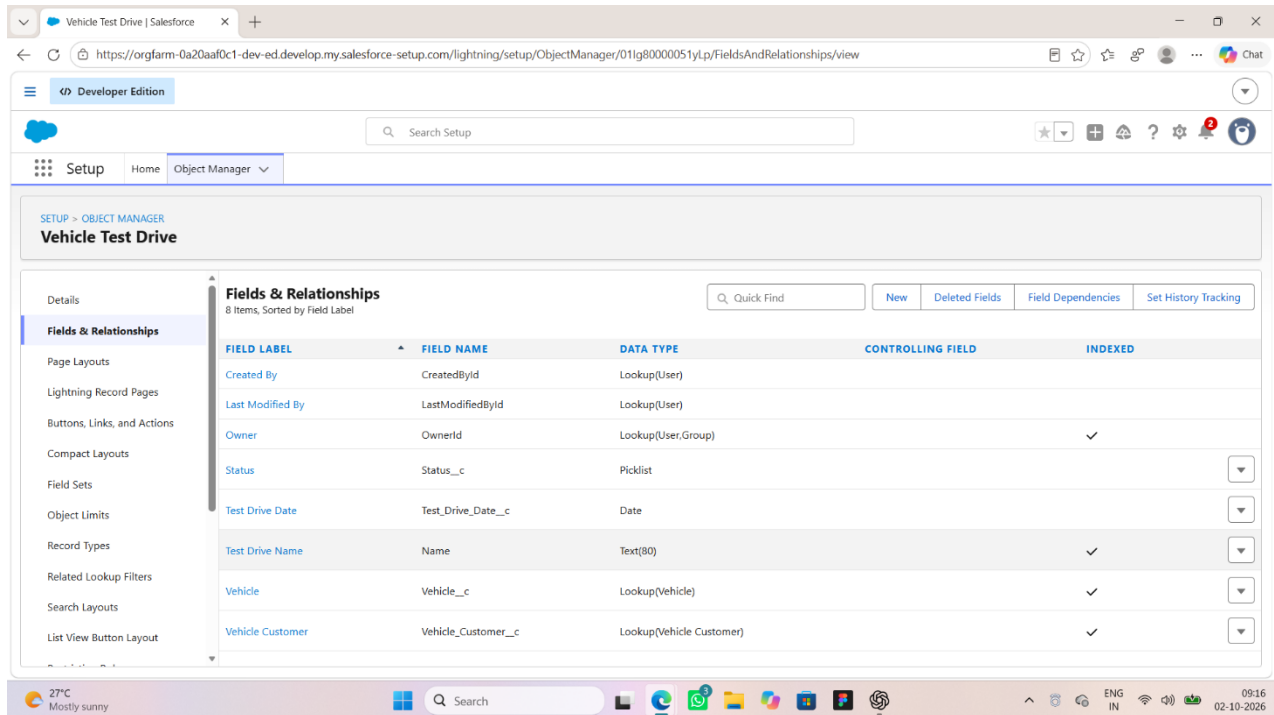
Explanation: The Vehicle Customer object's fields: Address, Email, Phone, Customer Name and Preferred Vehicle Type. Address is used for dealer assignment and Email for the test drive reminder.



5. Vehicle_Test_Drive__c (Custom Object)

Field	Data Type	Details
Vehicle_Customer__c	Lookup	Lookup to Vehicle_Customer__c
Vehicle__c	Lookup	Lookup to Vehicle__c
Test_Drive_Date__c	Date	Scheduled test drive date
Status__c	Picklist	Scheduled, Completed, Canceled

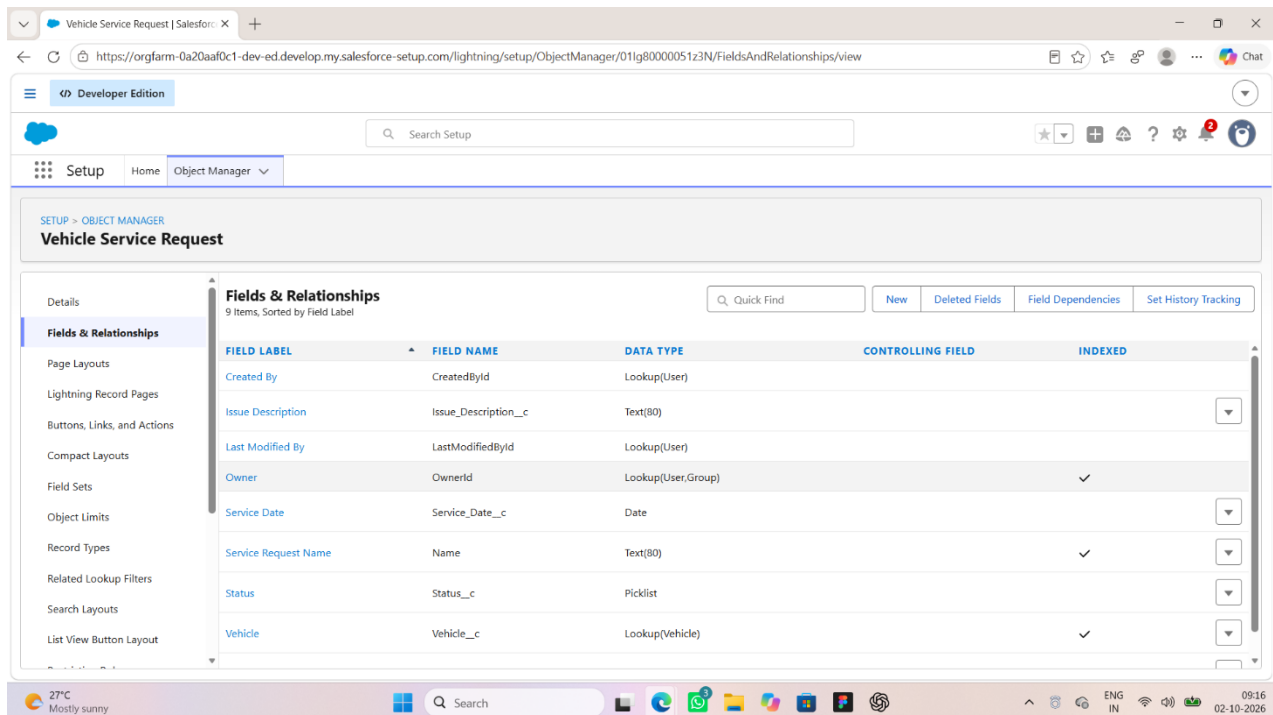
Explanation: The Vehicle Test Drive object's fields: Test Drive Date, Status (picklist), and lookups to Vehicle and Vehicle Customer. The reminder flow uses Test Drive Date and Status.



6. Vehicle_Service_Request__c (Custom Object)

Field	Data Type	Details
Vehicle_Customer__c	Lookup	Lookup to Vehicle_Customer__c
Vehicle__c	Lookup	Lookup to Vehicle__c
Service_Date__c	Date	Scheduled service date
Issue_Description__c	Text	Description of the issue
Status__c	Picklist	Requested, In Progress, Completed

Explanation: The Vehicle Service Request object's fields: Issue Description, Service Date, Status (picklist), and lookups to Vehicle and Vehicle Customer.



Object Field Creation – Picklist

1. Go to Setup → click **Object Manager** → type the object name (Vehicle) in the Quick Find bar → click the object.
2. Click **Fields & Relationships** → **New**.
3. Select the data type **Picklist** and click **Next**.
4. Field Label: **Vehicle Model**; fill in the picklist values (Sedan, SUV, EV, etc.).
5. Click **Next** → **Next** → **Save & New**.

Object Field Creation – Number

1. Repeat steps 1 and 2 from the previous activity.
2. Select the data type **Number** and click **Next**.
3. Field Label: **Stock Quantity**.
4. Click **Next** → **Next** → **Save & New**.

Relationship Field Creation – Lookup (Vehicle to Dealer)

1. Select the data type **Lookup Relationship** and click **Next**.
2. Select the **Vehicle Dealer** object in the Related To field and click **Next**.
3. Field Label: **Dealer**; Field Name: **Dealer**.
4. Click **Next** and **Save**.

More Fields Creation

Create the remaining fields as listed in the Key Fields tables above.

Milestone 6: Automation

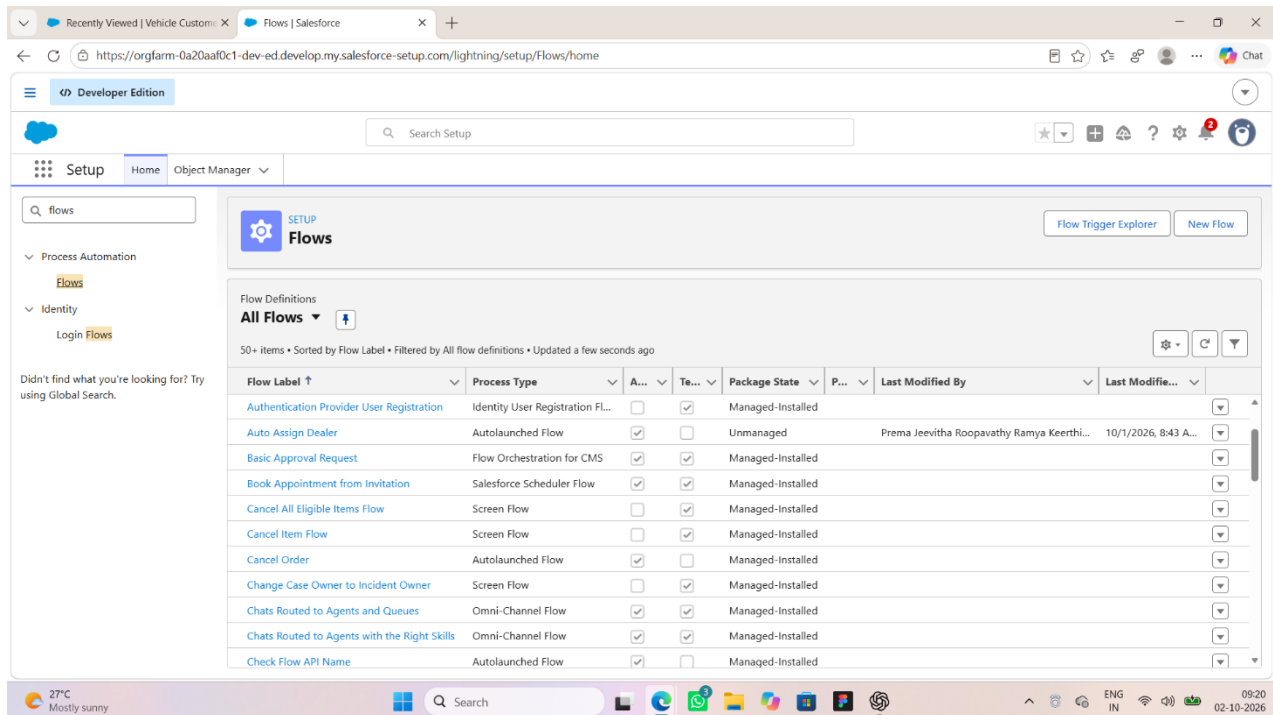
Flow 1: Record-Triggered Flow to Assign the Nearest Dealer

This flow assigns the nearest dealer according to the customer's location when a new vehicle order is created.

1. In Quick Find, type **Flows** and click **Flows**. Click **New Flow**.
2. Select **Start From Scratch** and click **Next**.
3. Select **Record-Triggered Flow** and click **Create**.
4. Select the **Vehicle Order** object. Trigger the flow when: **A record is created**. Set Entry Condition: All Conditions Are Met (AND) – Field **Status__c**, Operator **Equals**, Value **Pending**.
5. Click + → select **Get Records** and configure the elements in the table below.
6. Click + → select **Update Records** (see table below).
7. Click **Save**, give the label name **Auto Assign Dealer**.
8. Click **Activate** to activate the flow.

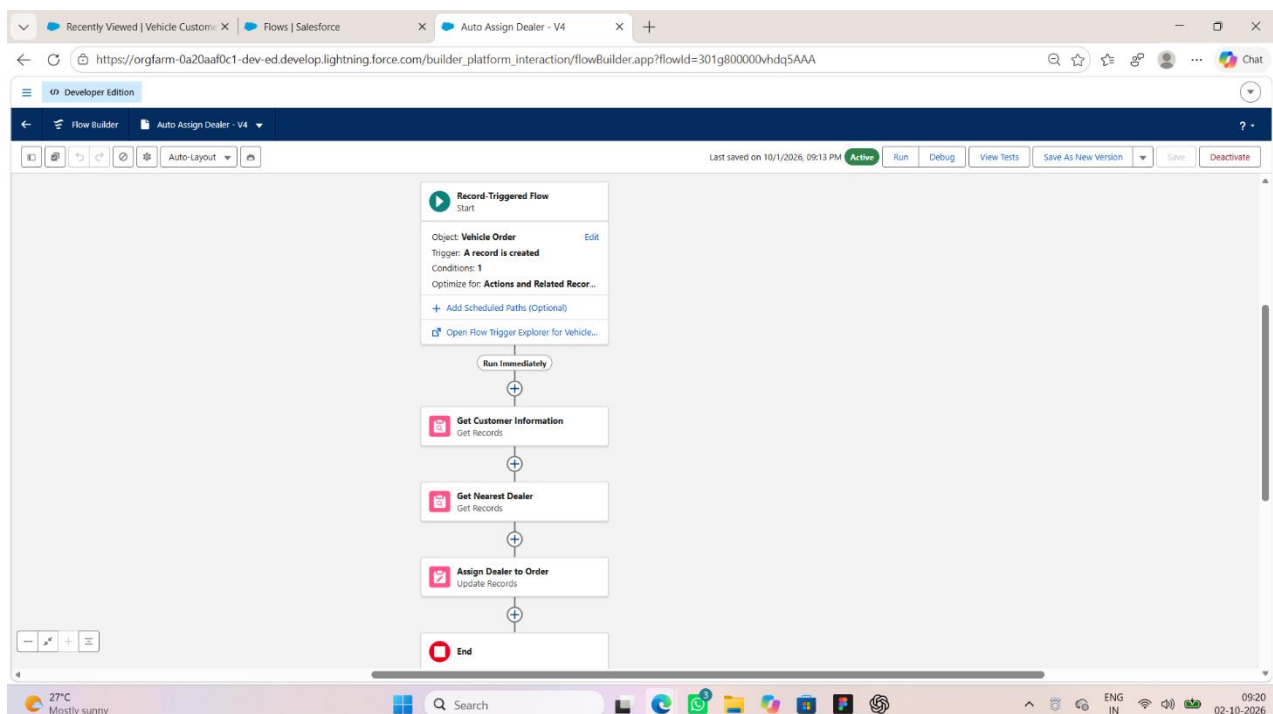
Element	Label	Configuration
Get Records	Get Customer Information	Object: Vehicle Customer Condition: Id Equals {!\$Record.Vehicle_Customer__c}
Get Records	Get Nearest Dealer	Object: Vehicle Dealer Condition: Dealer_Location__c Equals {!Get_Customer_Information.Address__c}
Update Records	Assign Dealer to Order	Use the IDs and all field values from a record or record collection Record(s) to update: {!Get_Nearest_Dealer}

Explanation: The Flows list in Setup shows the “Auto Assign Dealer” flow as an active, unmanaged autolaunched flow created in this org.



Flow Label	Process Type	A...	Te...	Package State	P...	Last Modified By	Last Modifie...
Authentication Provider User Registration	Identity User Registration Fl...	☐	☒	Managed-Installed			
Auto Assign Dealer	Autolaunched Flow	☒	☐	Unmanaged		Prema Jeevitha Roopavathy Ramya Keerthi...	10/1/2026, 8:43 A...
Basic Approval Request	Flow Orchestration for CMS	☒	☒	Managed-Installed			
Book Appointment from Invitation	Salesforce Scheduler Flow	☒	☒	Managed-Installed			
Cancel All Eligible Items Flow	Screen Flow	☐	☒	Managed-Installed			
Cancel Item Flow	Screen Flow	☐	☒	Managed-Installed			
Change Order	Autolaunched Flow	☒	☐	Managed-Installed			
Change Case Owner to Incident Owner	Screen Flow	☐	☒	Managed-Installed			
Chats Routed to Agents and Queues	Omni-Channel Flow	☒	☒	Managed-Installed			
Chats Routed to Agents with the Right Skills	Omni-Channel Flow	☒	☒	Managed-Installed			
Check Flow API Name	Autolaunched Flow	☒	☐	Managed-Installed			

Explanation: Flow Builder view of “Auto Assign Dealer”. It starts when a Vehicle Order record is created (with one entry condition), then runs Get Customer Information, Get Nearest Dealer and Assign Dealer to Order. The flow status is Active.



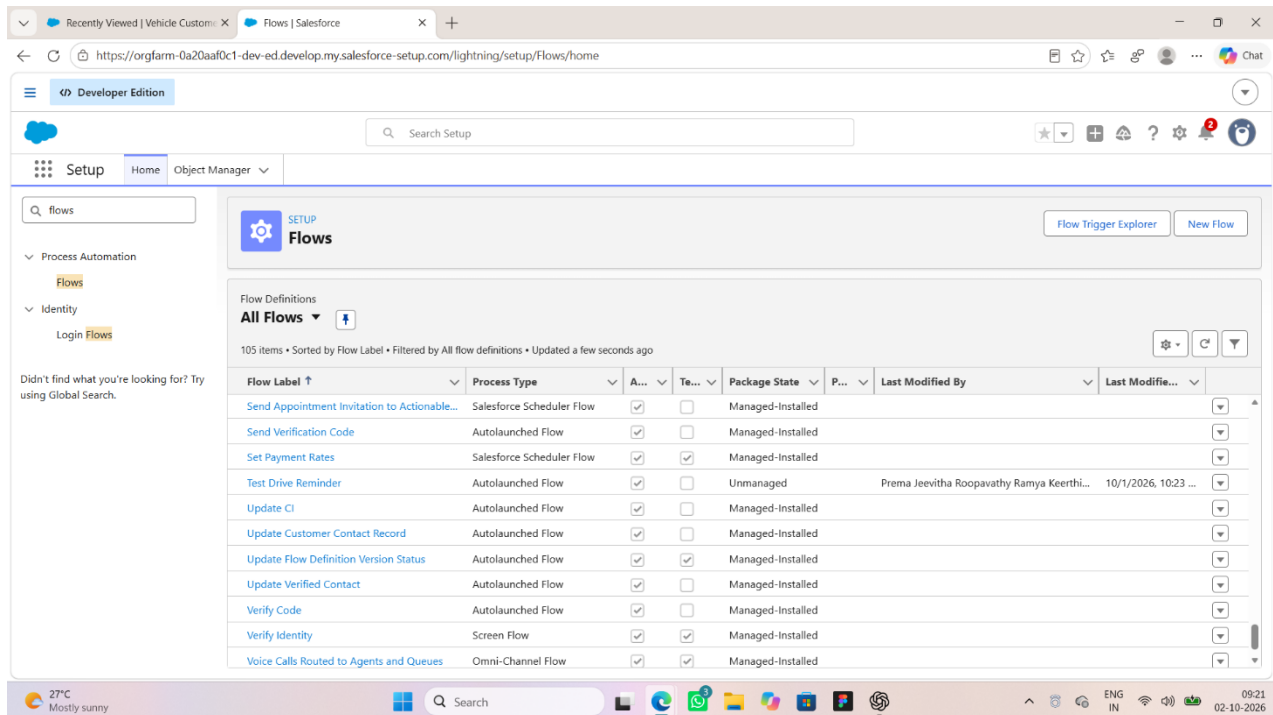
Flow 2: Record-Triggered Flow for Test Drive Email Reminders

This flow sends an email to the customer reminding them about the test drive.

1. Select **Record-Triggered Flow** and click **Create**.
2. Select the **Vehicle Test Drive** object. Trigger the flow when: **A record is created or updated**. Set Entry Condition: All Conditions Are Met (AND) – Field **Status__c**, Operator **Equals**, Value **Scheduled**.
3. Click + **Add Scheduled Paths** (below the trigger): Label **Reminder Before Test Drive**; Time Source **Test_Drive_Date__c**; Offset Number **1**; Offset Options **Days Before**. Click **Done**.
4. Click + **Add Element** → **Get Records**: Label **Get Customer Information**; Object **Vehicle_Customer__c**; Filter Condition: Id = {!\$Record.Vehicle_Customer__c}; store only the first record; automatically store all fields.
5. Send the reminder email: click + **Add Element** → **Action** → **Send Email** and configure it as shown in the table below.
6. Click **Save** with the label name **Test Drive Reminder**.
7. Click **Activate** to activate the flow.

Send Email Setting	Value
Label	Send Test Drive Reminder
Subject	“Reminder: Your Test Drive is Tomorrow!”
Recipient Address List	{!Get_Customer_Information.Email__c}
Rich-Text-Formatted Body	True
Body	Variable EmailSent (API Name: EmailSent)
Use Line Breaks	True

Explanation: The Flows list also shows the “Test Drive Reminder” flow, active and unmanaged, created in this org.



Flows

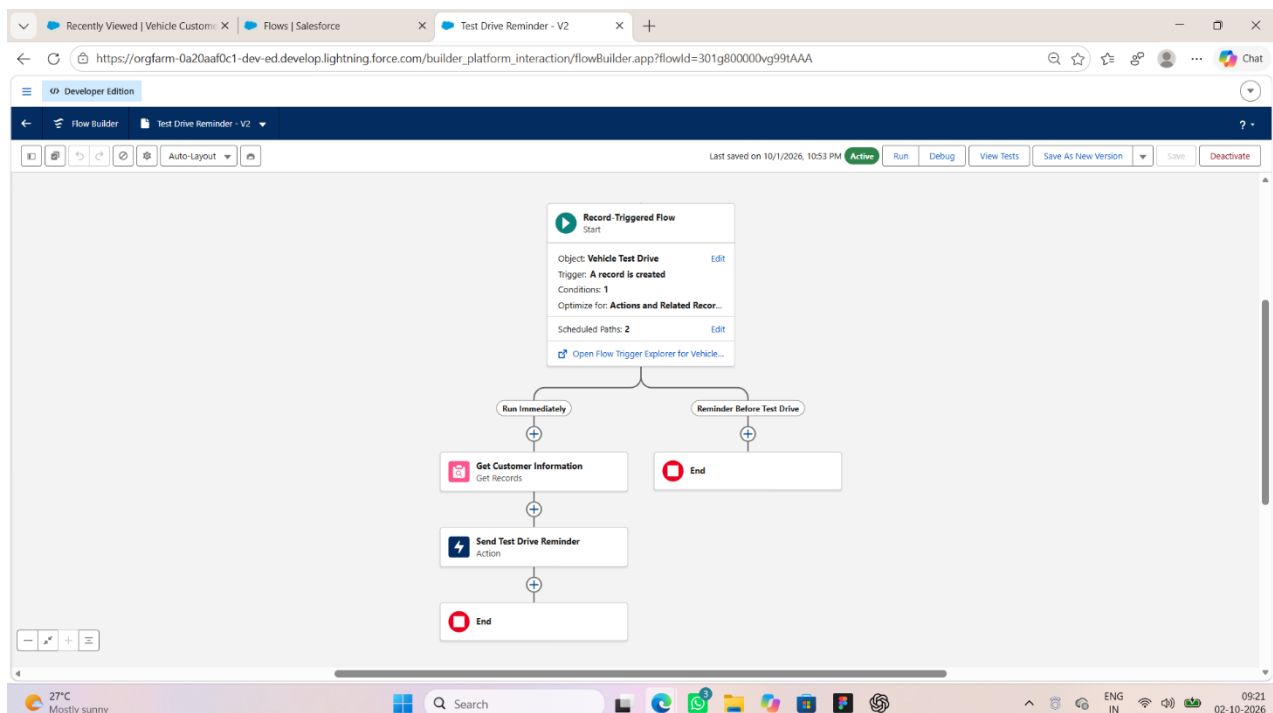
Flow Definitions

All Flows

105 items • Sorted by Flow Label • Filtered by All flow definitions • Updated a few seconds ago

Flow Label	Process Type	A...	Te...	Package State	P...	Last Modified By	Last Modifie...
Send Appointment Invitation to Actionable...	Salesforce Scheduler Flow	☒	☐	Managed-Installed			
Send Verification Code	Autolaunched Flow	☒	☐	Managed-Installed			
Set Payment Rates	Salesforce Scheduler Flow	☒	☒	Managed-Installed			
Test Drive Reminder	Autolaunched Flow	☒	☐	Unmanaged		Prema Jeevitha Roopavathy Ramya Keenthi...	10/1/2026, 10:23 ...
Update CI	Autolaunched Flow	☒	☐	Managed-Installed			
Update Customer Contact Record	Autolaunched Flow	☒	☐	Managed-Installed			
Update Flow Definition Version Status	Autolaunched Flow	☒	☒	Managed-Installed			
Update Verified Contact	Autolaunched Flow	☒	☐	Managed-Installed			
Verify Code	Autolaunched Flow	☒	☐	Managed-Installed			
Verify Identity	Screen Flow	☒	☒	Managed-Installed			
Voice Calls Routed to Agents and Queues	Omni-Channel Flow	☒	☒	Managed-Installed			

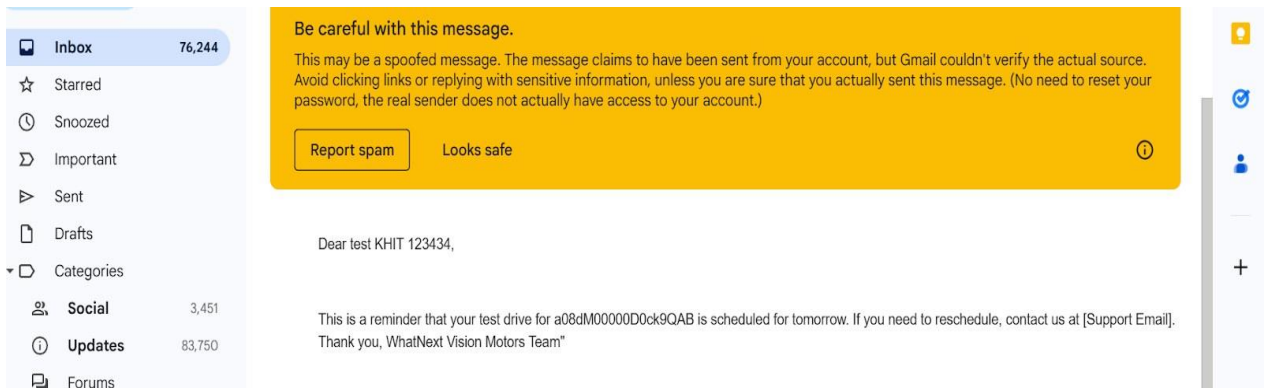
Explanation: Flow Builder view of “Test Drive Reminder”. The Run Immediately path gets the customer’s information and sends the reminder email. A scheduled path, “Reminder Before Test Drive”, is set to fire before the test drive date. The flow status is Active.



Test Your Flow

After activation, a Vehicle Test Drive record with the status Scheduled was created and the customer received the reminder email below.

Explanation: The reminder email received during testing. It tells the customer that the test drive is scheduled for tomorrow and is signed by the WhatNext Vision Motors Team.



Milestone 7: Apex and Batch Class

Create Apex and Trigger Batch Jobs

Step 1–4: Apex Handler Class

1. Click **Developer Console** from the gear icon.
2. Click **File** → **New** → **Apex Class**.
3. Give the Apex class the name **VehicleOrderTriggerHandler** and click **OK**.
4. Write the Apex code (source code below).

Source Code – VehicleOrderTriggerHandler:

```
public class VehicleOrderTriggerHandler {  
  
    public static void handleTrigger(List<Vehicle_Order__c> newOrders,  
                                     Map<Id, Vehicle_Order__c> oldOrders,  
                                     Boolean isBefore, Boolean isAfter,  
                                     Boolean isInsert, Boolean isUpdate) {  
  
        if (isBefore) {  
            if (isInsert || isUpdate) {  
                preventOrderIfOutOfStock(newOrders);  
            }  
        }  
        if (isAfter) {  
            if (isInsert || isUpdate) {  
                updateStockOnOrderPlacement(newOrders);  
            }  
        }  
    }  
}
```

```

// Method to prevent orders when the vehicle is out of stock
private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null) {
            vehicleIds.add(order.Vehicle__c);
        }
    }

    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c
                                   FROM Vehicle__c
                                   WHERE Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }

        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c <= 0) {
                    order.addError(
                        'This vehicle is out of stock. Order cannot be placed.'
                    );
                }
            }
        }
    }
}

// Method to update vehicle stock when an order is placed
private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
            vehicleIds.add(order.Vehicle__c);
        }
    }

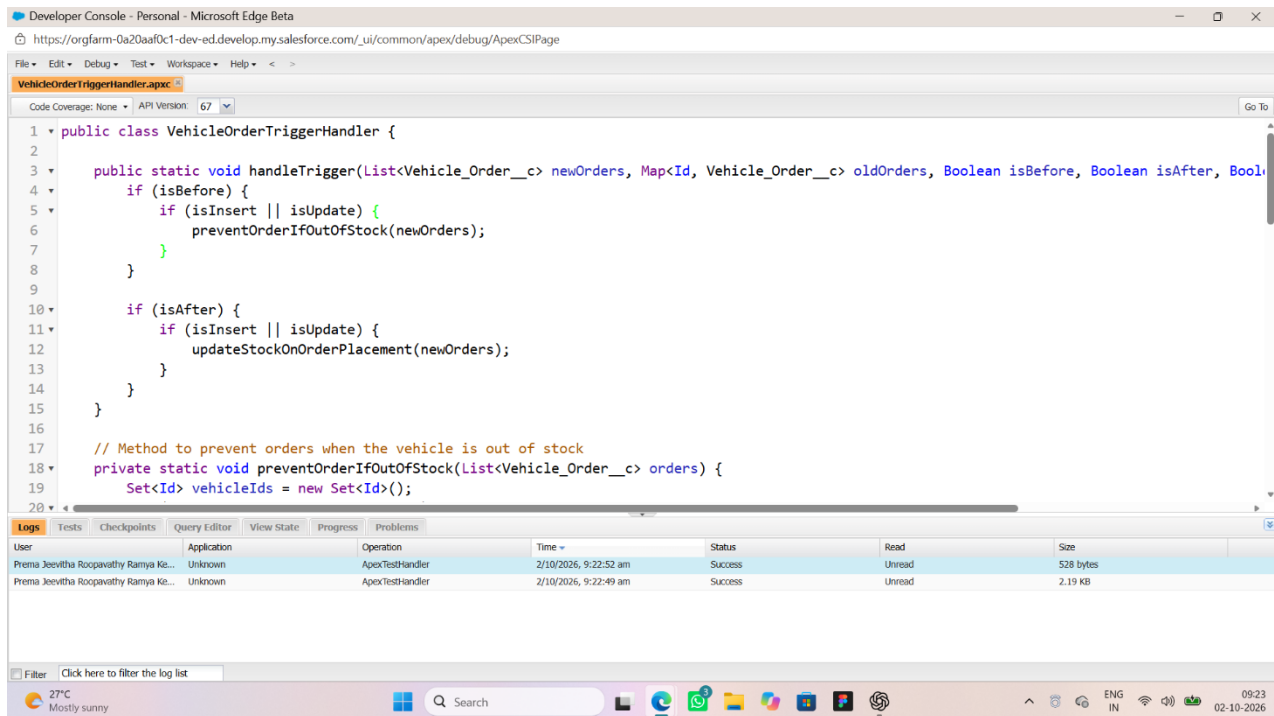
    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c
                                   FROM Vehicle__c
                                   WHERE Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }

        List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c > 0) {
                    vehicle.Stock_Quantity__c -= 1;
                    vehiclesToUpdate.add(vehicle);
                }
            }
        }

        if (!vehiclesToUpdate.isEmpty()) {
            update vehiclesToUpdate;
        }
    }
}

```

Explanation: The Developer Console showing the VehicleOrderTriggerHandler class. handleTrigger calls preventOrderIfOutOfStock before insert/update and updateStockOnOrderPlacement after insert/update. The log panel shows successful executions.



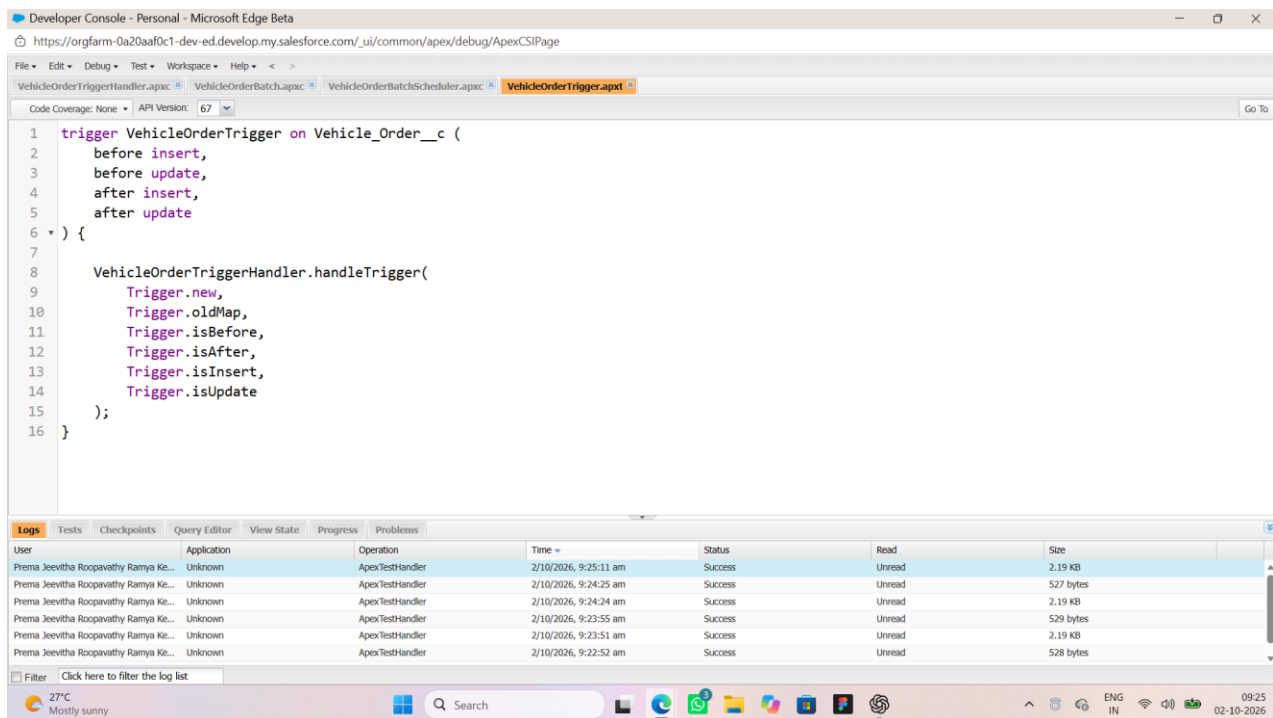
Step 5–7: Apex Trigger

1. Write the trigger handler: click **File** → **New** → **Apex Trigger**.
2. Enter the trigger name **VehicleOrderTrigger** and select the **Vehicle_Order__c** sObject.
3. Call the Apex class in the trigger (source code below).

Source Code – VehicleOrderTrigger:

```
trigger VehicleOrderTrigger on Vehicle_Order__c (
    before insert, before update, after insert, after update
) {
    VehicleOrderTriggerHandler.handleTrigger(
        Trigger.new,
        Trigger.oldMap,
        Trigger.isBefore,
        Trigger.isAfter,
        Trigger.isInsert,
        Trigger.isUpdate
    );
}
```

Explanation: The VehicleOrderTrigger on Vehicle_Order__c. It runs before and after insert and update, and passes the trigger context to VehicleOrderTriggerHandler.



Step 8: Create Batch Job

Scenario: a customer places an order, but the vehicle is out of stock.

- The order remains **Pending**.
- After new stock is added, the batch job updates the order to **Confirmed**.

Create a new Apex class named **VehicleOrderBatch**.

Source Code – VehicleOrderBatch:

```
global class VehicleOrderBatch implements Database.Batchable<SObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Vehicle__c, Status__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending'
            AND Vehicle__c != null
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();

        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
```

```

        vehicleIds.add(order.Vehicle__c);
    }

    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c
                                   FROM Vehicle__c
                                   WHERE Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }

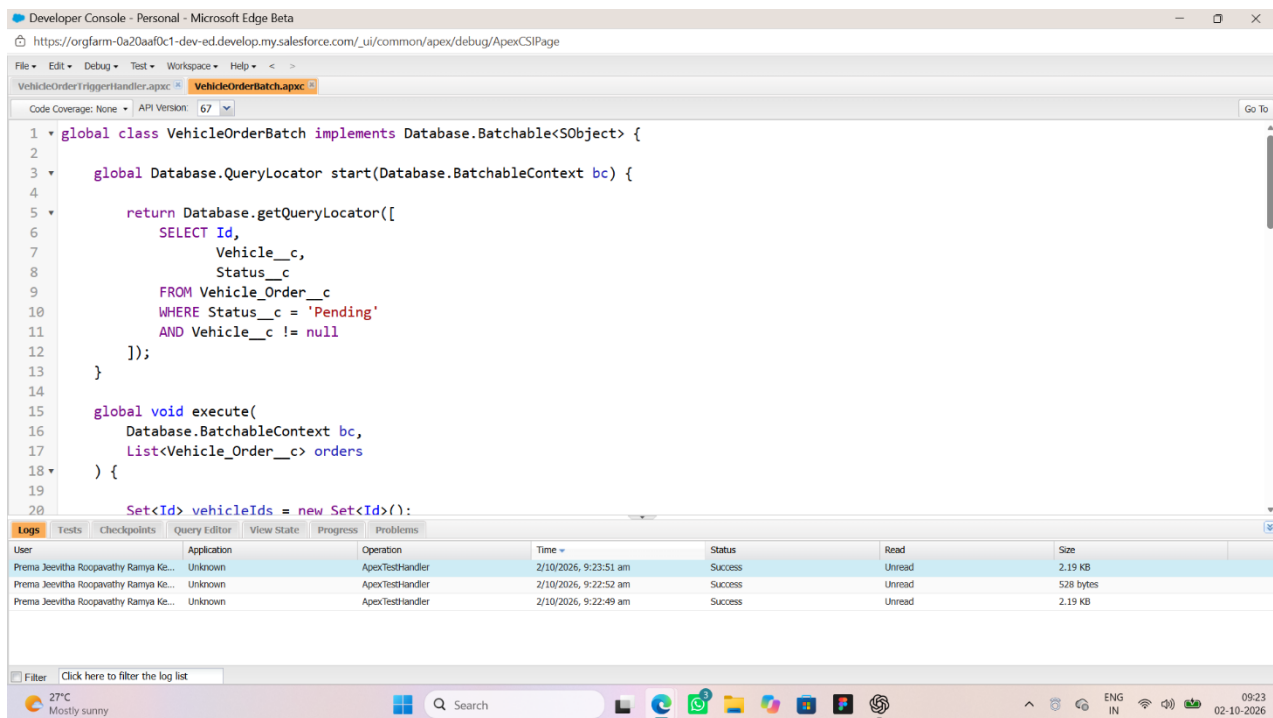
        List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
        List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c > 0) {
                    order.Status__c = 'Confirmed';
                    vehicle.Stock_Quantity__c -= 1;
                    ordersToUpdate.add(order);
                    vehiclesToUpdate.add(vehicle);
                }
            }
        }

        if (!ordersToUpdate.isEmpty()) {
            update ordersToUpdate;
        }
        if (!vehiclesToUpdate.isEmpty()) {
            update vehiclesToUpdate;
        }
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle order batch job completed.');
```

Explanation: The VehicleOrderBatch class. Its start method selects Pending orders that have a vehicle, so the batch can confirm them once stock is available.



Step 9: Create Schedule Class

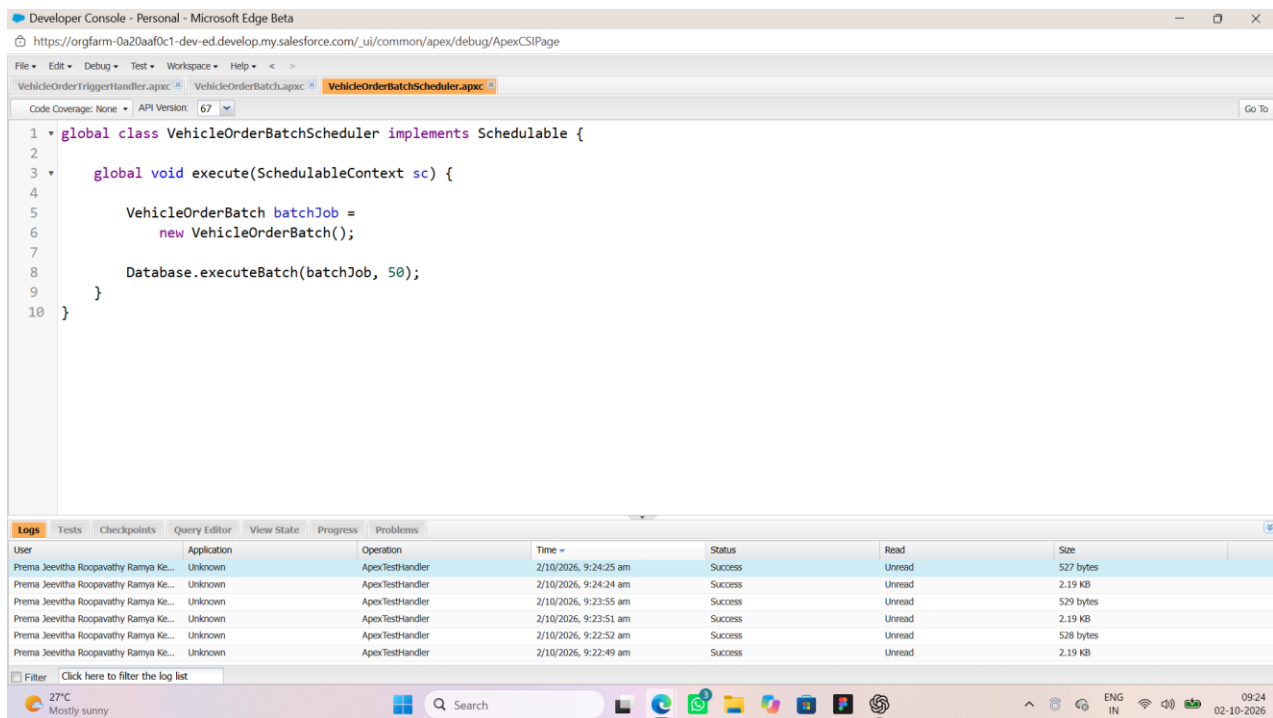
Create a Schedule class named **VehicleOrderBatchScheduler** that invokes the batch class.

Source Code – VehicleOrderBatchScheduler:

```
global class VehicleOrderBatchScheduler implements Schedulable {

    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 is the batch size
    }
}
```

Explanation: The VehicleOrderBatchScheduler class. It implements Schedulable and runs VehicleOrderBatch with a batch size of 50.



Step 10: Schedule the Batch Job

To run the batch job every night at midnight, open **Debug** → **Open Execute Anonymous Window** and execute the following code:

Source Code – Schedule Command:

```
String cronExp = '0 0 0 * * ?'; // Runs daily at 12:00 AM (midnight)
System.schedule('Daily Vehicle Order Processing', cronExp,
    new VehicleOrderBatchScheduler());
```

Step 11: Check Where the Scheduled Job Is Running

Go to Setup → type **Jobs** in Quick Find → **Scheduled Jobs**. The job **Daily Vehicle Order Processing** is listed as a Scheduled Apex job.

Explanation: The Scheduled Jobs page lists “Daily Vehicle Order Processing” as a Scheduled Apex job, with its next run at 12:00 AM. This confirms the batch job is scheduled to run every night.

Scheduled Jobs

All Scheduled Jobs

The All Scheduled Jobs page lists all of the jobs scheduled by your users. Multiple job types may display on this page. You can delete scheduled jobs if you have the permission to do so.

Percentage of Scheduled Jobs Used: 2%
You have currently used 2 scheduled Apex jobs out of an allowed organization limit of 100 active or scheduled jobs. To learn about how this limit is calculated and what contributes to it see the [Lightning Platform Apex Limits](#) topic.

View: [All Scheduled Jobs](#) [Create New View](#)

Action	Job Name	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	Daily Inventory Sync	Roopavathy Ramya Keerthika, Prema Jeevitha	10/1/2026, 7:46 PM		10/2/2026, 12:00 AM	Scheduled Apex	08eg800000LV8FX
Manage Del Pause Job	Daily Vehicle Order Processing	Roopavathy Ramya Keerthika, Prema Jeevitha	10/1/2026, 7:27 PM		10/2/2026, 12:00 AM	Scheduled Apex	08eg800000LVTQ9
Del	Metalytics Data Loader Job for Org : 00Dy800000L8GV3	User Integration	9/24/2026, 8:30 AM	10/1/2026, 5:38 PM	10/2/2026, 5:38 PM	Autonomous Data Loader Job	08eg800000KQYYr
	Program Milestone Computation Cron Job	Process, Automated	9/24/2026, 8:30 AM	10/1/2026, 5:01 PM	10/1/2026, 11:59 PM	Program Milestone Computation Cron Job	08eg800000KQYYp
	Program Status Update Cron Job	Process, Automated	9/24/2026, 8:30 AM	10/1/2026, 8:01 PM	10/2/2026, 5:00 AM	Program Status Update Cron Job	08eg800000KQYYq

7. FUNCTIONAL & PERFORMANCE TESTING

During testing, screenshots were captured from Salesforce to validate vehicle creation, dealer assignment, stock validation triggers, order status updates, batch processing, scheduled reminders, and dashboard analytics. All features were verified to ensure correct functionality and performance.

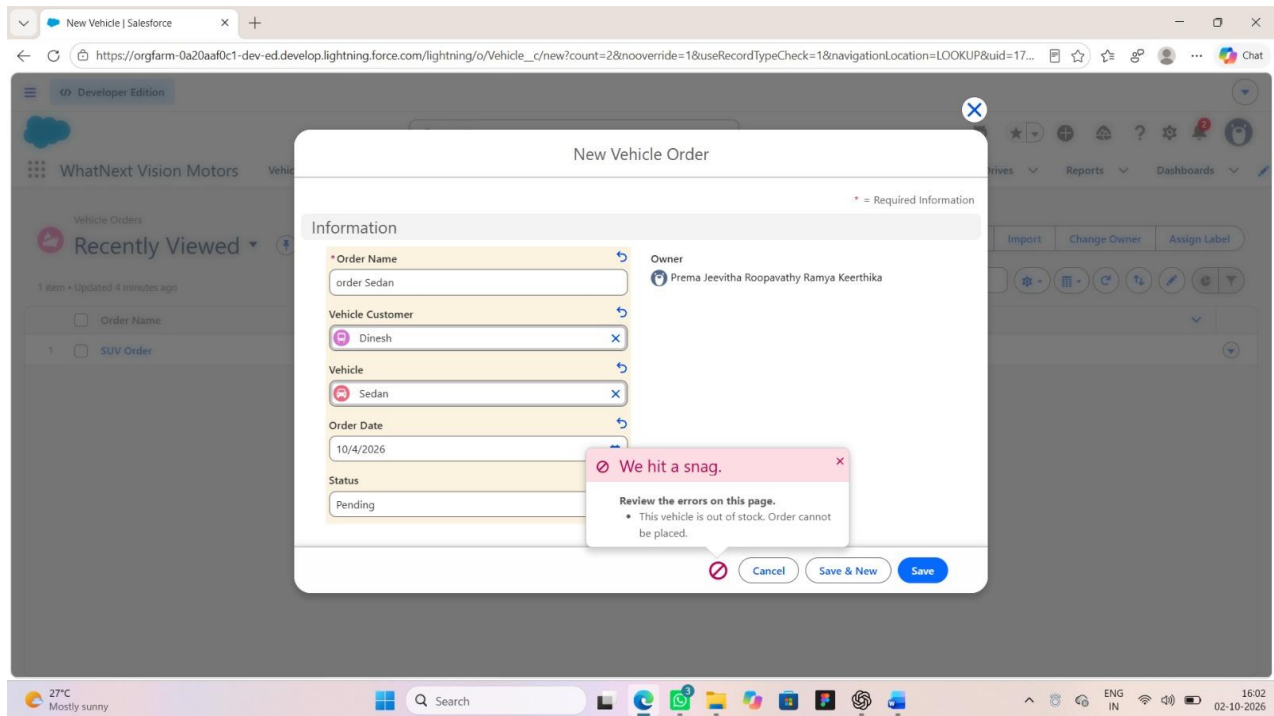
Test Summary

S.NO	Feature Tested	Test Condition	Expected Result	Result
1	Data model	Six custom objects with fields and relationships created	Objects deployed and fields available	Verified
2	Navigation	Lightning App “WhatNext Vision Motors” with module tabs	All modules accessible from the app	Verified
3	Dealer assignment	New Vehicle Order created with status Pending	“Auto Assign Dealer” flow runs automatically	Verified
4	Stock validation	Order placed for a vehicle with stock quantity of 0	Order blocked with an out-of-stock error message	Verified
5	Order status update	Pending order after new stock is added	Batch job updates the order to Confirmed	Verified
6	Batch processing	VehicleOrderBatch executed through the scheduler (batch size 50)	Apex executions complete successfully	Verified
7	Scheduled processing	Daily Vehicle Order Processing scheduled	Job appears in Scheduled Jobs with next run at 12:00 AM	Verified
8	Scheduled reminders	Test drive with status Scheduled	Reminder email sent to the customer one day before	Verified
9	Dashboard analytics	Reports and Dashboards tabs in the app	Order status, dealer performance and sales metrics available	Verified

Stock Validation Test:

When an order was saved for a vehicle that is out of stock, the trigger blocked the transaction and displayed the validation message “This vehicle is out of stock. Order cannot be placed.”

Explanation: Test result: when a user saves an order for a vehicle with no stock, the trigger blocks the save and shows “This vehicle is out of stock. Order cannot be placed.”



Performance Observations:

- Apex execution logs in the Developer Console indicate that each process completed successfully without execution errors.
- Batch operations handle 50 records per batch through the scheduled process, allowing nightly order processing to be completed efficiently.
- Stock availability is checked during the before-insert and before-update trigger stages, ensuring that orders with insufficient stock are blocked immediately.

8. ADVANTAGES & DISADVANTAGES:

Advantages:

- **Unified information management** – vehicle, dealer, customer, order, test drive, and service request details can be maintained within a single Salesforce platform.
- **Process automation** – tasks such as dealer allocation, inventory verification, order status changes, and customer reminders are handled automatically.
- **Improved monitoring** – dashboards and reports provide useful insights into order progress, dealer activities, and sales performance.

Disadvantages:

- **Platform reliance** – the application is closely dependent on Salesforce and its available features and services.
- **Setup requirements** – the initial implementation requires time and effort to configure objects, fields, automation flows, and Apex components.

9. CONCLUSION

The Salesforce CRM solution implemented for WhatNext Vision Motors transforms traditional vehicle sales and dealer management processes into a more organized and automated system. By reducing manual tasks and improving process accuracy, the platform supports smoother operations and provides customers with a more convenient and satisfying experience.

10. FUTURE SCOPE

- Mobile-based vehicle ordering application
- GPS-enabled dealer location and navigation integration
- Automated SMS alerts and customer notifications
- AI-driven vehicle demand prediction and forecasting

11. APPENDIX

Item	Reference
Source Code	Salesforce Configuration – VehicleOrderTriggerHandler, VehicleOrderTrigger, VehicleOrderBatch, VehicleOrderBatchScheduler (Milestone 7)
Dataset	Salesforce Objects – Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive, Vehicle Service Request
GitHub / Demo	Salesforce Developer Org

Thank You